

Robust and Efficient Penetration-Free Elastodynamics without Barriers

JUNTIAN ZHENG, Carnegie Mellon University, USA

ZHAOFENG LUO, Carnegie Mellon University, USA

MINCHEN LI, Carnegie Mellon University, USA and Genesis AI, USA



Fig. 1. **Squishy balls under extreme compression.** Five elastic squishy balls are compressed by a moving boundary to extreme stress, generating dense contact contacts, and then released to rebound. The scene contains 2.61M DoFs, 2.25M tetrahedra, and generates up to 1.45M active contact constraints. With significantly fewer Newton iterations and better conditioning, we achieve a 98.5 $\times$ speedup over GIPC [Huang et al. 2024], averaging 5.37 s per frame.

We introduce a barrier-free optimization framework for non-penetration elastodynamic simulation that matches the robustness of Incremental Potential Contact (IPC) while overcoming its two primary efficiency bottlenecks: (1) reliance on logarithmic barrier functions to enforce non-penetration constraints, which leads to ill-conditioned systems and significantly slows down the convergence of iterative linear solvers; and (2) the time-of-impact (TOI) locking issue, which restricts active-set exploration in collision-intensive scenes and requires a large number of Newton iterations. We propose a novel second-order constrained optimization framework featuring a custom augmented Lagrangian solver that avoids TOI locking by immediately incorporating all requisite contact pairs detected via CCD, enabling more efficient active-set exploration and leading to significantly fewer Newton iterations. By adaptively updating Lagrange multipliers rather than increasing penalty stiffness, our method prevents stagnation at zero TOI while maintaining a well-conditioned system. We further introduce a constraint filtering and decay mechanism to keep the active set compact and stable, along with a theoretical justification of our method’s finite-step termination and first-order time integration accuracy under a cumulative TOI-based termination criterion. A comprehensive set of experiments demonstrates the efficiency, robustness, and accuracy of our method. With a GPU-optimized simulator design, our method achieves an up to 103 $\times$ speedup over GIPC on challenging, contact-rich benchmarks – scenarios that were previously tractable only with barrier-based methods. *Our code and data will be open-sourced.*

CCS Concepts: • **Computing methodologies** → **Physical simulation.**

Additional Key Words and Phrases: Finite Element Method, Elastodynamics, Collision Handling, Constrained Optimization, Active Set Method

1 INTRODUCTION

In recent years, Incremental Potential Contact (IPC) [Li et al. 2020a] has pioneered the penetration-free simulation of nonlinear elastic solids, offering guaranteed algorithmic convergence, solution accuracy, and minimal tuning of algorithmic parameters. IPC has been successfully applied to simulate a range of challenging phenomena. However, its computational efficiency remains a key bottleneck in time-sensitive applications such as robotics and virtual reality, even with recent GPU-accelerated variants featuring highly optimized Gauss–Newton and preconditioned conjugate gradient (PCG) solvers [Huang et al. 2024, 2025a].

We identify two major sources of inefficiency in IPC: (1) the use of logarithmic barrier functions leads to severely *ill-conditioned* systems, requiring many PCG iterations to solve; and (2) IPC suffers from the *TOI locking* issue [Lan et al. 2023] in collision-intensive scenarios, where its filtered line search severely slows down active set exploration, a process that is inherently combinatorially complex in inequality-constrained optimization.

The *TOI locking* problem arises because each Newton update is truncated by the smallest time-of-impact (TOI) detected via continuous collision detection (CCD) [Li et al. 2021; Wang et al. 2021], causing the earliest contact to stall the entire optimization step (Figure 2). As a result, many iterations are needed to progressively discover and incorporate all relevant contact pairs into the constraint set. Recent Gauss–Seidel-type strategies [Chen et al. 2025; Lan et al. 2023] attempt to mitigate this issue using local updates, but

Authors’ Contact Information: Juntian Zheng, juntianz@andrew.cmu.edu, Carnegie Mellon University, Pittsburgh, PA, USA; Zhaofeng Luo, zhaofen2@andrew.cmu.edu, Carnegie Mellon University, Pittsburgh, PA, USA; Minchen Li, minchernl@gmail.com, Carnegie Mellon University, Pittsburgh, PA, USA and Genesis AI, San Carlos, CA, USA.

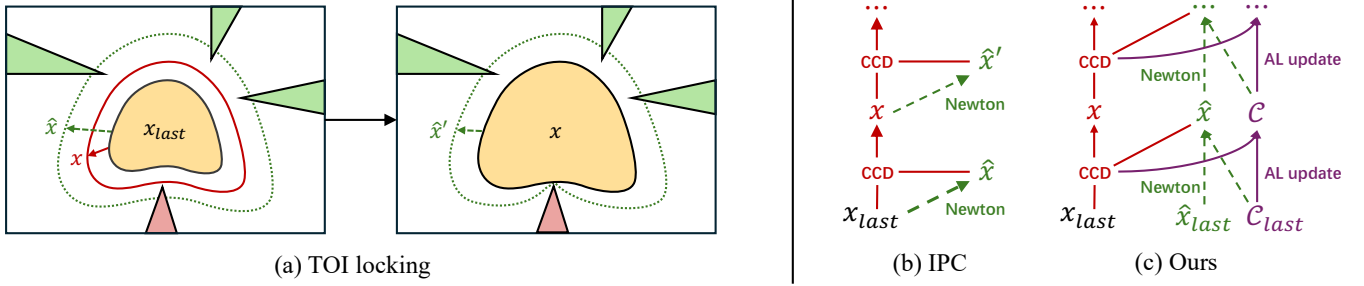


Fig. 2. **Tackling the TOI locking issue.** (a) The advancement in each Newton iteration is stalled at the contact pair with the minimum TOI (marked in red), thus incorporating only the earliest contacts into the constraint set (assuming small contact radius). All other contacts (marked in green) can continue to block the CCD in subsequent iterations. (b) IPC’s Newton iterations, discarding the intermediate states $\hat{x}$. (c) Our modified framework with explicitly maintained intermediate state $\hat{x}$ and constraint set C carrying Lagrange multipliers, updated by a primal–dual augmented Lagrangian solver.

remain limited in scenarios involving high stiffness or large deformations, due to their reliance on sublinearly convergent coordinate descent methods.

In this work, we propose a novel alternative that retains super-linearly convergent Newton iterations while *improving active set exploration efficiency*. Our method immediately incorporates all requisite contacts detected by CCD into subsequent iterations, allowing earlier response to all these potential contacts. The key insight stems from reexamining IPC’s CCD-truncated Newton updates as shown in Figure 2: in each iteration, a possibly penetrating state $\hat{x}$ is generated from the previous penetration-free iterate x_{last} , after which IPC applies CCD to obtain a new penetration-free state x by truncating the path between x_{last} and $\hat{x}$. IPC then discards $\hat{x}$ and proceeds from x , potentially losing information of all contact pairs with larger TOI at $\hat{x}$. Instead, we resume Newton iterations directly from $\hat{x}$, allowing contacts with larger TOI to generate immediate responses.

This shift necessitates abandoning unsigned distances, whose gradients reverse upon penetration in $\hat{x}$. We thus switch to using signed distances by linearizing the unsigned distance function at the last penetration-free iterate x_{last} during each update, which provides consistent and generalizable local contact force even in codimensional settings. This immediately disables the use of the log-barrier, since it is undefined at a penetrating state $\hat{x}$ with negative distances. A seemingly straightforward alternative is to replace the barrier with a penalty-based collision response defined on the penetration depths in $\hat{x}$. However, constructing an effective penalty energy in this setting is nontrivial. Simple approaches such as a naïve quadratic penalty fail to guarantee that CCD-truncated Newton iterations will make consistent progress in challenging cases: even with a positive contact offset and large penalty stiffness, direct penalty methods can result in persistent penetrations across successive iterates of $\hat{x}$, causing CCD to repeatedly return zero TOI and thereby stalling progress (see §6.3). To prevent such stagnation, a mechanism is required to strengthen contact response for persistent penetrations without indefinitely increasing the stiffness, which would otherwise degrade system conditioning just as the logarithmic barrier does.

To address this, we design a custom augmented Lagrangian (AL) solver [Nocedal and Wright 2006], which augments the penalty

energy with iteratively updated Lagrange multipliers to provide improved control over constraint satisfaction. This enables persistent penetrations to be resolved by progressively adjusting the Lagrange multipliers without stiffening the penalty energy. As a result, the constraint violations converge toward a positive contact offset, enabling CCD to produce nonzero TOI and ensuring continued progress. This also yields *better-conditioned systems* than the barrier-based approaches, significantly reducing the number of PCG iterations required even when using a simple block-Jacobi preconditioner (see §6.4).

The use of linearized distance functions presents another key challenge: if the constraint set is not carefully managed, it may accumulate unnecessary and potentially conflict constraints, making the system overconstrained or even infeasible to solve [Li et al. 2020a]. Accordingly, we introduce a novel filtering scheme to avoid adding spatially irrelevant or redundant constraints and a decay mechanism to gradually phase out inactive ones. These techniques ensure that the constraint set remains compact and evolves smoothly across iterations, enabling efficient computation and preventing instability or oscillatory behavior in contact resolution.

To further improve efficiency without compromising accuracy, we employ a termination criterion based on the cumulative TOI, similar to heuristics used in [Ando 2024; Wang et al. 2023]. Beyond its practical effectiveness, we provide a theoretical guarantee that our method satisfies this criterion with tight tolerance in finite steps while achieving first-order accuracy as a time integrator. In practice, our method consistently achieves larger average TOIs compared to IPC (see §6.3), resulting in significantly fewer iterations to satisfy the same accuracy requirement. As a result, our method also tolerates looser termination thresholds without suffering from the damping artifacts commonly observed in IPC when limiting iteration counts.

Building upon these innovations, we present a novel elastodynamic simulator for penetration-free contact, achieving up to 103.15× speedup over GIPC [Huang et al. 2024] in challenging, contact-intensive benchmarks that were previously tractable only by barrier-based methods. On moderate scenarios, our simulator (in double precision) achieves a 5.05× speedup over Cubic Barrier [Ando 2024] and a 33.1× speedup over OGC [Chen et al. 2025] using sufficient iterations to avoid artifacts, despite both baselines operating in single

precision. In more challenging scenarios, our method outperforms Cubic Barrier by up to 84.4X, while OGC suffers from severe artifacts even when spending orders-of-magnitude more computation time than ours. These significant improvements are made possible by the following technical contributions:

- A novel 2nd-order constrained optimization framework (§4.1), with a primal-dual augmented Lagrangian solver (§4.2) that ensures well-conditioned systems and consistent convergence progress, and a constraint filtering and decay mechanism (§4.3) to achieve fast and robust active-set exploration.
- Theoretical justification that the TOI-based termination criterion ensures finite-step termination and first-order time integration accuracy in our method (§4.4). We also demonstrate that this criterion can accelerate other CCD-based contact solvers, such as IPC.
- A GPU-optimized high-performance simulator design, featuring novel techniques for accelerated assembly of analytic elasticity Hessians under SPD projection (§5.1), conditioning-aware adjustment of penalty stiffness (§5.2), and penalty-free enforcement of moving boundary conditions (§5.3).

Our supplementary videos and materials are available at <https://github.com/wiso-enoji/Barrier-Free-Supplementary>. The code and data will be released publicly.

2 RELATED WORK

2.1 Barrier-free Collision Handling

We categorize all approaches not employing a diverging barrier function into the class of barrier-free methods. A straightforward way to handle collisions is through the *penalty-based* methods [Armero and Petőcz 1998; Benson and Hallquist 1990; Chen et al. 2024; Kim and Eberle 2020; Wriggers 1995], which introduce an energy term penalizing the penetration depth of geometric primitives into one another. Since the penalty energy is non-zero only when penetrations are present, the penalty-based methods inevitably allow penetrations in order to generate collision response. Even with a positive contact offset used to separate contacting primitives, the penalty stiffness must grow significantly to prevent penetration under high stress (see §6.3).

Another traditional approach to collision handling, commonly used in cloth simulation, is the *impact zone* method [Bridson et al. 2002; Harmon et al. 2008; Li et al. 2020b; Narain et al. 2012; Provot 1997; Tang et al. 2018], which groups penetrating primitive pairs into connected regions after solving the dynamics and projects each region back to a non-penetrating state. The projection is typically formulated as a linearly constrained optimization problem, solved using LCP solvers or first-order iterative methods. Despite their efficiency, impact zone methods lack the guarantee that all penetrations can be resolved within a finite number of iterations, which may lead to failure in highly complex contact configurations commonly encountered in elastodynamics simulations.

Fictitious domain methods (also known as air-mesh methods) offer an alternative approach that introduces a separate discretization of the void space and enforce global injectivity by maintaining non-negative volumes in both the material and air elements to handle contact [Jiang et al. 2017; Misztal and Bærentzen 2012; Müller

et al. 2015]. While this formulation provides a unified geometric constraint preventing inversion, it suffers from severe distortion-induced locking artifacts [Fang et al. 2021], necessitating frequent remeshing which can be highly inefficient in 3D.

Another class of collision handling approaches [Daviet et al. 2011; Harmon et al. 2008; Jean and Moreau 1992; Kane et al. 1999; Kaufman et al. 2008, 2014; Macklin et al. 2019; Otaduy et al. 2009; Verschoor and Jalba 2019] is based on *sequential quadratic programming* (SQP), which formulates each time step as an optimization problem with non-penetration constraints, whose solution is approximated by sequentially solving a series of quadratic programming subproblems. The common pipeline of SQP-based methods involves, at each iteration, collecting the constraint set via discrete collision detection (DCD), forming a quadratic program with the quadratically approximated objective and linearized constraints, and solving it using LCP, conjugate residual (CR), Newton-like methods, or black-box QP solvers. The main limitations lie in the efficiency of solving the QP subproblems and the large number of iterations required to achieve a penetration-free state, which may even be impossible for complex scenarios under large time steps, as benchmarked in [Li et al. 2020a].

It is noteworthy that some barrier-free methods are designed to generate penetration-free trajectories by applying strict collision detection to filter the advancing steps. Impact zone methods can be equipped with CCD to ensure that the iterations terminate upon reaching a penetration-free state [Li et al. 2020b; Tang et al. 2018]. Wang et al. [2023] modified the CCD-based impact zone method by relaxing the linear penetration-free path constraint to a piecewise linear path and replacing CCD with a DCD-based displacement upper bound to ensure penetration-free iteration steps. Lan et al. [2024] introduced a PD framework with a CCD safeguard that dynamically adjusts contact constraint weights according to number of iterations they remain active. Compared to the barrier-based methods, their lack of convergence guarantees may cause the iterations to stall in challenging elastodynamics scenarios with complex collision or high-speed impact, in which case early termination is required for the simulation to progress, thereby sacrificing accuracy. In comparison, our method is guaranteed to generate penetration-free results in a finite number of steps with 1st-order time integration accuracy.

2.2 Barrier-based Methods

A more recent class of collision handling methods draws inspiration from the interior-point method, which introduces the logarithmic barrier function to ensure that the iterates remain within the feasible region (see [Nocedal and Wright 2006]). The pioneering work of Li et al. [2020a] proposed Incremental Potential Contact (IPC), which achieves penetration-free large deformation elastodynamics through a C^2 -continuous locally supported barrier function incorporated in the projected Newton framework with a CCD-truncated line search. A series of follow-up works have extended IPC to codimensional elements [Li et al. 2021], rigid-body systems [Chen et al. 2022; Ferguson et al. 2021; Lan et al. 2022a], granular impact dynamics [Jiang et al. 2022], and coupling with MPM- [Li et al. 2022, 2024] and SPH-based [Xie et al. 2023] materials. Li et al. [2023b]

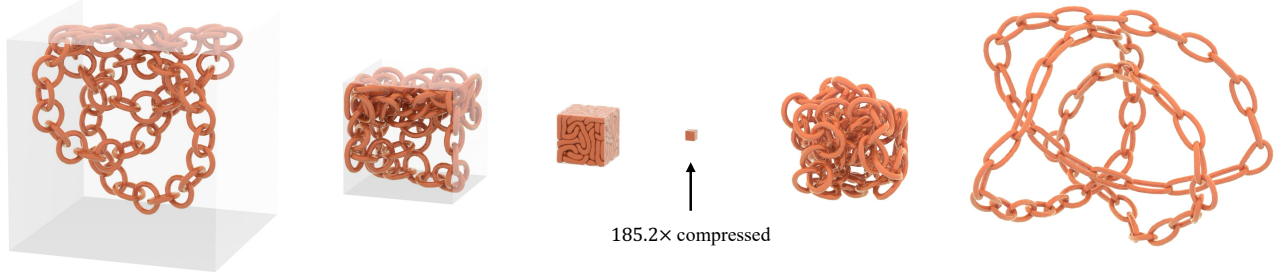


Fig. 3. **Compressing chain rings.** Three nested elastic rings are compressed within a shrinking boundary and then released to rebound, reaching a density increase of up to 185.2 $\times$ during compression. The simulation remains stable and preserves topology under extreme deformation and complex contacts.

proposed a variational formulation of IPC’s contact energies, identified discretization errors that lead to artificially distorted level sets near mesh edges and nodes, and introduced preliminary solutions to mitigate these issues.

Despite the improved accuracy and robustness in handling large deformations and complex contact scenarios, the main limitation of IPC lies in its low computational efficiency. A group of works have aimed to enhance its simulation efficiency by introducing various geometry representation, including medial elastics [Lan et al. 2021], subspace DoFs [Du et al. 2025; Trusty et al. 2024], and higher-order finite elements [Ferguson et al. 2023; Meng et al. 2025], to reduce the system DoFs. Another line of research focuses on improving the parallelization of IPC on modern GPUs. Huang et al. [2024] proposed GIPC, a fully GPU-optimized IPC framework with a parallelization-friendly Gauss-Newton approximation of the IPC barrier. Du et al. [2024] presented another GPU-parallelized IPC framework supporting efficient rigid-soft contact for robot manipulation. More recently, Huang et al. [2025a] further accelerated IPC for stiff materials by improving the Multilevel Additive Schwarz (MAS) [Wu et al. 2022] linear solver preconditioner through connectivity enhancement and introducing a faster Hessian matrix assembly strategy for the affine-deformable coupled system.

Besides the standard projected Newton, the IPC barrier is also compatible with various optimization methods, including preconditioned nonlinear conjugate gradient [Shen et al. 2024] and projective dynamics [Lan et al. 2022b; Li et al. 2023a]. Lan et al. [2023] proposed a stencil descent method for IPC contact, which performs parallelized local hybrid Gauss-Seidel and Jacobi updates, and ensures penetration-free results via local-global CCD filtering. Chen et al. [2025] smoothly extended the IPC barrier using a quadratic function and corrected its behavior under large contact radius through an offset geometry-based distance calculation, followed by energy optimization via displacement-bound vertex block descent [Chen et al. 2024] updates. These methods mostly rely on low-order convergent optimizers to trade accuracy of the momentum equation for efficiency, which can easily lead to artificially damped motions. In contrast, Guo et al. [2024] retains the Newton’s method for superlinear convergence and introduces Lagrangian terms for contact pairs with small distances to improve system conditioning. However, their formulation still relies on logarithmic barrier functions, and

its efficiency remains sensitive to the tuning of multiple algorithmic parameters.

In addition to the logarithmic barrier, other types of barrier functions have been proposed to handle penetration-free contact, including the non-local tangent-point energy [Strzelecki and von der Mosel 2013] used in geometry processing [Sassen et al. 2024], and the recently proposed locally supported Geometric Contact Potential [Huang et al. 2025b] that avoids the spurious forces generated by the IPC barrier under large contact radius. Ando [2024] introduces a cubic contact energy that adaptively adjusts the stiffness based on the elasticity Hessian and distance gap, exhibiting barrier-like behavior as the contact distance approaches zero.

3 BACKGROUND

We begin by introducing the essential background and notation appearing throughout our framework. We focus on simulating the dynamic behavior of a set of elastic bodies spatially discretized into linear tetrahedral elements. The system’s state consists of the stacked nodal positions $\mathbf{x} \in \mathbb{R}^{3N}$ and velocities $\mathbf{v} \in \mathbb{R}^{3N}$, in which N denotes the number of nodes after discretization. The continuous-time trajectories $\mathbf{x}(t), \mathbf{v}(t)$ are further discretized into timesteps $\mathbf{x}^t, \mathbf{v}^t$. In the implicit Euler time integration commonly used in physics-based animation, the discrete timesteps are updated by

$$\begin{aligned}\mathbf{x}^{t+1} &= \mathbf{x}^t + h\mathbf{v}^{t+1}, \\ \mathbf{v}^{t+1} &= \mathbf{v}^t + h\mathbf{M}^{-1}(\mathbf{f}_{\text{int}}(\mathbf{x}^{t+1}) + \mathbf{f}_{\text{ext}}^{t+1}),\end{aligned}\tag{1}$$

in which h denotes the timestep size, $\mathbf{M}$ denotes the lumped mass matrix, $\mathbf{f}_{\text{int}}$ and $\mathbf{f}_{\text{ext}}$ denote the total internal and external forces, respectively. Let $U(\mathbf{x})$ be the total potential energy associated with the internal forces (i.e., $\mathbf{f}_{\text{int}}(\mathbf{x}) = -\partial U / \partial \mathbf{x}$), including the smoothed friction energy as in IPC (detailed in Appendix A). By introducing the Incremental Potential [Kane et al. 2000]

$$E(\mathbf{x}, \tilde{\mathbf{x}}) = \frac{1}{2}(\mathbf{x} - \tilde{\mathbf{x}})^T \mathbf{M}(\mathbf{x} - \tilde{\mathbf{x}}) + h^2 U(\mathbf{x}),\tag{2}$$

where $\tilde{\mathbf{x}} = \mathbf{x}^t + h\mathbf{v}^t + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}^t$, we can reformulate the nonlinear system (Equation 1) into an equivalent minimization problem over $E(\mathbf{x}, \tilde{\mathbf{x}})$ w.r.t. $\mathbf{x}$, followed by velocity updates.

When an intersection-free trajectory is required, positive distance constraints are imposed on every intermediate state during the transition between adjacent timesteps [Li et al. 2020a], i.e., there

exists a path $\mathbf{L}_{\mathbf{x}^t \rightarrow \mathbf{x}^{t+1}}$ connecting $\mathbf{x}^t$ and $\mathbf{x}^{t+1}$ such that

$$\mathbf{d}(\mathbf{x}) > 0, \forall \mathbf{x} \in \mathbf{L}_{\mathbf{x}^t \rightarrow \mathbf{x}^{t+1}}, \quad (3)$$

where $\mathbf{d} = \{d_i\}_{i \in \mathcal{I}}$ is the set of unsigned distances between all vertex-face and edge-edge pairs. Inspired by the interior-point method, Li et al. [2020a] proposed IPC by adding smoothly clamped log barriers to the incremental potential to handle the constraints:

$$B(\mathbf{x}) = E(\mathbf{x}, \tilde{\mathbf{x}}) + \kappa \sum_{i \in \mathcal{I}} b(d_i(\mathbf{x}), \hat{d}), \quad (4)$$

where $\kappa, \hat{d}$ are the barrier parameters, and $b(d, \hat{d})$ is a barrier function supported on $d \in (0, \hat{d})$ that diverges as $d \rightarrow 0$. An intersection-free trajectory is then ensured by iteratively minimizing quadratic proxy of $B(\mathbf{x})$ using line search with projected Newton while clamping the search directions using CCD.

Despite its robustness, accuracy, and penetration-free guarantees, IPC exhibits significant efficiency bottlenecks. These primarily stem from the clamping of search directions and the severe ill-conditioning induced by the sharp logarithmic barrier functions.

4 METHOD

4.1 Overview

Problem Formulation. Instead of relying on the log-barrier function as in IPC, we explore an alternative approach to advance $\mathbf{x}^t$ to the next timestep along an intersection-free trajectory. Rather than using the proximity distances to generate the contact constraints, we explicitly maintain an active constraint set $\mathcal{C}$ including all primitive pairs potentially leading to intersections, enabling earlier contact response during the solve for more effective iterations. In each timestep, we divide the time-stepping into a sequence of subproblems with linear inequality constraints:

$$\hat{\mathbf{x}}^{[k]} = \arg \min_{\tilde{\mathbf{x}}} E(\hat{\mathbf{x}}, \tilde{\mathbf{x}}) \quad \text{s.t.} \quad \mathbf{c}^{[k]}(\hat{\mathbf{x}}) \geq 0, \quad (5)$$

in which $\mathbf{c}^{[k]}$ is a set of linearized contact constraints generated from the current active set $\mathcal{C}^{[k]}$ (see §4.2). In each iteration k , we perform one or several Newton steps to obtain an inexact solution of $\hat{\mathbf{x}}^{[k]}$ before proceeding to the next iteration.

Constructing Penetration-Free Paths. The active sets $\mathcal{C}^{[k]}$ are progressively updated along with the intermediate states $\hat{\mathbf{x}}^{[k]}$, and may not contain all necessary constraints in the early iterations. Due to the incompleteness of $\mathcal{C}^{[k]}$ and the linearization of the constraints, $\{\hat{\mathbf{x}}^{[k]}\}_{k \geq 1}$ are *not guaranteed to be intersection-free*. Based on the intermediate states, we further construct an intersection-free piecewise linear path $\{\mathbf{x}^{[k]}\}_{k \geq 0}$ starting from $\mathbf{x}^{[0]} = \mathbf{x}^t$, where each $\mathbf{x}^{[k+1]}$ is a linear interpolation between $\mathbf{x}^{[k]}$ and $\hat{\mathbf{x}}^{[k+1]}$:

$$\mathbf{x}^{[k+1]} = (1 - \alpha^{[k+1]})\mathbf{x}^{[k]} + \alpha^{[k+1]}\hat{\mathbf{x}}^{[k+1]}, \quad 0 \leq \alpha^{[k+1]} \leq 1, \quad (6)$$

such that no intersections occur during the linear transition from $\mathbf{x}^{[k]}$ to $\mathbf{x}^{[k+1]}$, which can be realized using CCD. This decoupling of intersection-free states and solver iterates resembles IPC's quadratic proxy minimization followed by filtered line search. However, rather than discarding the proxy solution, we leverage it to anticipate potential contact constraints and to guide the estimation of optimization progress, as detailed later.

Termination Criterion. It turns out that each $\mathbf{x}^{[k]}$ is also a linear combination of the previous timestep $\mathbf{x}^t$ and the intermediate states $\{\hat{\mathbf{x}}^{[i]}\}_{i=1}^k$:

$$\mathbf{x}^{[k]} = \beta_0^{[k]}\mathbf{x}^t + \sum_{i=1}^k \beta_i^{[k]}\hat{\mathbf{x}}^{[i]} \quad (7)$$

with coefficients

$$\beta_i^{[k]} = \alpha^{[i]} \prod_{j=i+1}^k (1 - \alpha^{[j]}), \quad 0 \leq i \leq k, \quad (8)$$

where $\alpha^{[0]} = 1$. When the step sizes $\alpha^{[k]}$ are sufficiently large, the influence of the initial state $\mathbf{x}^t$ and early intermediate states $\hat{\mathbf{x}}^{[1]}, \hat{\mathbf{x}}^{[2]}, \dots$ on $\mathbf{x}^{[k]}$ decays exponentially. Consequently, as we iteratively update the active sets $\mathcal{C}^{[k]}$ and the intermediate states $\hat{\mathbf{x}}^{[k]}$ while maintaining non-infinitesimal $\alpha^{[k]}$, we eventually reach an intersection-free state $\mathbf{x}^{[k]}$ that is primarily composed of later iterates. Since these later iterates are solved using more complete $\mathcal{C}^{[k]}$, they better capture the true contact interactions, making $\mathbf{x}^{[k]}$ more accurate. We terminate the solve by setting $\mathbf{x}^{t+1} = \mathbf{x}^{[k]}$ and computing $\mathbf{v}^{t+1}$ once the total weight of the first $K_{\min}$ intermediate states falls below a small user-defined threshold ϵ :

$$\sum_{i=0}^{K_{\min}-1} \beta_i^{[k]} < \epsilon. \quad (9)$$

As we will demonstrate later, this criterion enables early termination while avoiding damping artifacts and maintaining reasonable accuracy for our method.

Algorithm 1 outlines the main pipeline of our time-stepping solver. Similar to IPC, MAXSTEPsize computes a conservative step size $\alpha^{[k+1]}$ that ensures intersection-free (and inversion-free for non-invertible materials such as Neo-Hookean) trajectories. The two core subroutines, SOLVESUBPROBLEM and UPDATEACTIVSET, are detailed in the following subsections.

Challenges. Several key challenges may arise within this framework. First, efficiently solving the subproblem (Equation 5) remains challenging. To ensure large TOI when using the solved search direction, the approximate solution of $\hat{\mathbf{x}}^{[k]}$ needs to have small or even no constraint violations. Second, the termination of this pipeline relies on the values of $\alpha^{[k]}$ being sufficiently large, which is directly related to the update strategy of the active set $\mathcal{C}^{[k]}$. A conservative strategy (e.g., not updating at all) can easily lead to getting stuck at $\alpha^{[k]} = 0$ forever. Meanwhile, overly aggressive strategies may result in unnecessarily large active sets, thereby increasing computational cost and system instability (as the constraints are linearized). Finally, it is still questionable whether our TOI-based termination criterion is sufficiently strong, i.e., if it ensures the necessary order of convergence to the PDE solution as a time integrator.

To address these challenges, §4.2 introduces an efficient yet effective solver for Equation 5 based on the Augmented Lagrangian formulation. Section 4.3 presents our active set update strategy, and Section 4.4 discusses termination guarantees and our framework's first-order accuracy.

Algorithm 1 Main pipeline of our time-stepping algorithm.

Input: last timestep's state $\mathbf{x}^t, \mathbf{v}^t$ and active set C^t .
Output: current timestep's state $\mathbf{x}^{t+1}, \mathbf{v}^{t+1}$ and active set C^{t+1} .

- 1: $\tilde{\mathbf{x}} \leftarrow \mathbf{x}^t + h\mathbf{v}^t + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}^t$;
- 2: $\mathbf{x}^{[0]}, \hat{\mathbf{x}}^{[0]}, C^{[0]} \leftarrow \mathbf{x}^t, \mathbf{x}^t, C^t$;
- 3: Handle moving boundaries in $\hat{\mathbf{x}}^{[0]}$; ▷ See §5.3
- 4: $\beta^{[0]}, k \leftarrow 1, 0$; ▷ $\beta^{[k]}$ stores $\sum_{i=0}^{K_{\min}-1} \beta_i^{[k]}$
- 5: **while** $\beta^{[k]} > \epsilon$ **do**
- 6: $\hat{\mathbf{x}}^{[k+1]} \leftarrow \text{SOLVESUBPROBLEM}(\tilde{\mathbf{x}}, \mathbf{x}^{[k]}, \hat{\mathbf{x}}^{[k]}, C^{[k]});$
- 7: ▷ Algorithm 2, §4.2
- 8: $C^{[k+1]} \leftarrow \text{UPDATEACTIVETSET}(\mathbf{x}^{[k]}, \hat{\mathbf{x}}^{[k]}, C^{[k]});$
- 9: ▷ Algorithm 3, §4.3
- 10: $\alpha^{[k+1]} \leftarrow \text{MAXSTEPsize}(\mathbf{x}^{[k]}, \hat{\mathbf{x}}^{[k+1]});$ ▷ CCD queries
- 11: $\mathbf{x}^{[k+1]} \leftarrow (1 - \alpha^{[k+1]})\mathbf{x}^{[k]} + \alpha^{[k+1]}\hat{\mathbf{x}}^{[k+1]}$;
- 12: **if** $k + 1 \geq K_{\min}$ **then**
- 13: $\beta^{[k+1]} \leftarrow (1 - \alpha^{[k+1]})\beta^{[k]}$;
- 14: **else**
- 15: $\beta^{[k+1]} \leftarrow \beta^{[k]}$;
- 16: **end if**
- 17: $k \leftarrow k + 1$;
- 18: **end while**
- 19: **return** $\mathbf{x}^{[k]}, (\mathbf{x}^{[k]} - \mathbf{x}^t)/h, C^{[k]}$;

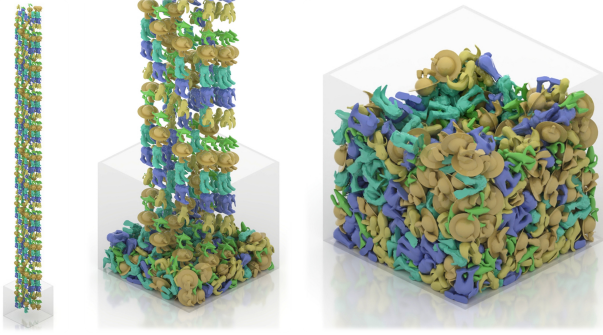


Fig. 4. **Animal well.** A challenging test case featuring a large collection of objects (1.34M tetrahedra) and high velocity induced by gravity. The topmost objects are accelerated to 19.7 m/s when colliding with the lower ones, traveling farther than their average size within a single time step.

4.2 Augmented Lagrangian-based Subproblem Solver

Constraint Linearization. We first formulate the subproblem (Equation 5) by defining the linear constraints $\mathbf{c}(\hat{\mathbf{x}})$. For each vertex-face or edge-edge pair indexed by i , the unsigned distance $d_i(\mathbf{x})$ between two primitives can be linearized into a signed distance function by first-order Taylor expansion at an intersection-free state $\mathbf{x}$:

$$\mathbf{c}(\hat{\mathbf{x}}) = \{c_i(\hat{\mathbf{x}}) := d_i(\mathbf{x}) + \nabla d_i(\mathbf{x})^T(\hat{\mathbf{x}} - \mathbf{x}) - \delta\}_{i \in I_C}, \quad (10)$$

where I_C denotes all primitive pairs in the current active set C and δ is the surface separation parameter similar to $\hat{d}$ in IPC. Expanding at $\mathbf{x}$ ensures correct orientation of the linearized constraints.

Augmented Lagrangian Formulation. We then design a customized Augmented Lagrangian method by introducing explicit Lagrange

multipliers to enforce constraints accurately without relying on excessively stiff penalty terms.

Specifically, we first introduce a set of non-negative slack variables $\mathbf{s} = \{s_i \in [0, \infty)\}_{i \in I_C}$ to transform the inequality constraints $c_i(\hat{\mathbf{x}}) \geq 0$ into equality constraints

$$c_i(\hat{\mathbf{x}}) - s_i = 0, \quad (11)$$

and then explicitly maintain a set of estimated Lagrange multipliers λ_i and add a Lagrangian term and a penalty term to the IP:

$$\mathcal{L} := E(\hat{\mathbf{x}}, \hat{\mathbf{s}}) + \sum_{i \in I_C} \gamma_i \left(\frac{\mu}{2} (c_i(\hat{\mathbf{x}}) - s_i)^2 - \lambda_i (c_i(\hat{\mathbf{x}}) - s_i) \right). \quad (12)$$

Here, μ is the penalty stiffness and γ_i are the decay factors to be discussed later. The standard Augmented Lagrangian method guarantees that if we alternate between minimizing $\mathcal{L}$ w.r.t. the primal variables $(\hat{\mathbf{x}}, \hat{\mathbf{s}})$ and updating the multipliers via

$$\lambda_i \leftarrow \lambda_i - \mu(c_i(\hat{\mathbf{x}}) - s_i), \quad (13)$$

then with a reasonably large fixed μ , the multipliers λ_i will eventually converge to the true Lagrange multipliers satisfying the KKT conditions [Nocedal and Wright 2006]. Subsequently, $\hat{\mathbf{x}}$ will also converge to the solution of Equation 5 up to a small contact gap δ with $c_i(\hat{\mathbf{x}}) - s_i$ tending to zero.

Alternating Primal Solve. However, jointly optimizing $(\hat{\mathbf{x}}, \hat{\mathbf{s}})$ remains a challenging inequality constrained optimization problem. In addition, without sufficiently accurate λ_i , solving this problem to high accuracy may be unnecessary. After exploration, we observed that a more practical strategy is sufficient: we further alternate between optimizing the slack variables via

$$s_i \leftarrow \max(0, c_i(\hat{\mathbf{x}}) - \lambda_i/\mu) \quad (14)$$

and updating $\hat{\mathbf{x}}$ by projected Newton iterations with line search to ensure decreasing total energy $\mathcal{L}$. We terminate this inner iteration when a full step is taken in the line search for $\hat{\mathbf{x}}$. For any elasticity linear in $\hat{\mathbf{x}}$, it always terminates in a single iteration. For nonlinear and non-convex elasticities, it effectively prevents λ_i from blowing up when $\hat{\mathbf{x}}$ becomes trapped at irregular landscapes, while still terminating in one iteration in most cases.

Constraint Decay. Algorithm 2 shows our solver for the subproblem (Equation 5). Line 8 and 9 compute the gradient and Hessian of the augmented penalty terms, which can be easily derived from Equation 12. Notably, in our Newton update for the primal variables, the constraints with $s_i > 0$ will not affect the gradient $\mathbf{G}$ since $c_i(\hat{\mathbf{x}}) - \lambda_i/\mu - s_i = 0$, but they will still generate non-zero terms in the Hessian $\mathbf{H}$. For these constraints in C that are actually inactive, the additional Hessian terms may slow down the convergence and performance of the subproblem solver. However, instantly removing these constraints from C will easily lead to oscillation and stability issues, as we will show in §6.3. Our solution is to introduce the decay factors γ_i to smoothly reduce the influence of the inactive constraints and restore them once they become active again. Specifically, γ_i is multiplied by a decay factor Γ per iteration when the constraint is inactive, scaling down the contribution of the augmented Lagrangian terms in Equation 12. Constraints that remain inactive for multiple iterations are eventually removed, as detailed

Algorithm 2 SolveSubproblem

Input: inertia target $\tilde{\mathbf{x}}$, intersection-free state $\mathbf{x}$, initial guess $\hat{\mathbf{x}}^0$, current active set $C = \{(i, \lambda_i, \gamma_i)\}_{i \in \mathcal{I}_C}$.

Output: approximate solution $\hat{\mathbf{x}}$ for Equation 5.

```

1:  $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}}^0$ ;
2: while true do
3:    $\mathbf{G} \leftarrow \nabla_{\hat{\mathbf{x}}} E(\hat{\mathbf{x}}, \tilde{\mathbf{x}})$ ;
4:    $\mathbf{H} \leftarrow \text{SPDPROJECT}(\nabla_{\hat{\mathbf{x}}}^2 E(\hat{\mathbf{x}}, \tilde{\mathbf{x}}))$ ;
5:   for all  $(i, \lambda_i, \gamma_i) \in C$  do
6:     Linearize the unsigned distance using  $d_i(\mathbf{x})$  and  $\nabla d_i(\mathbf{x})$ ;
7:      $s_i \leftarrow \max(0, c_i(\hat{\mathbf{x}}) - \lambda_i/\mu)$ ;
8:      $\mathbf{G} \leftarrow \mathbf{G} + \mu \gamma_i (c_i(\hat{\mathbf{x}}) - \lambda_i/\mu - s_i) \nabla d_i(\mathbf{x})$ ;
9:      $\mathbf{H} \leftarrow \mathbf{H} + \mu \gamma_i \nabla d_i(\mathbf{x}) \nabla d_i(\mathbf{x})^T$ ;
10:  end for
11:   $\mathbf{p} \leftarrow -\mathbf{H}^{-1} \mathbf{G}$ ; ▷ PCG Solve
12:   $r \leftarrow \text{LINESEARCH}(\hat{\mathbf{x}}, \mathbf{p})$ ; ▷  $\mathcal{L}(\hat{\mathbf{x}} + r\mathbf{p}) < \mathcal{L}(\hat{\mathbf{x}}), 0 \leq r \leq 1$ 
13:   $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + r\mathbf{p}$ ;
14:  if  $r = 1$  then
15:    break;
16:  end if
17: end while
18: for all  $(i, \lambda_i, \gamma_i) \in C$  do
19:    $s_i \leftarrow \max(0, c_i(\hat{\mathbf{x}}) - \lambda_i/\mu)$ ;
20:   if  $s_i = 0$  then
21:      $\lambda_i \leftarrow \lambda_i - \mu c_i(\hat{\mathbf{x}})$ ; ▷ Equivalent to Equation 13
22:      $\gamma_i \leftarrow \Gamma \gamma_i$ ;
23:   else
24:      $\lambda_i \leftarrow 0$ ; ▷ Equivalent to Equation 13
25:      $\gamma_i \leftarrow 1$ ;
26:   end if
27: end for
28: return  $\hat{\mathbf{x}}$ ;

```

in the next subsection. We use $\Gamma = 0.9$ in all experiments, which we find to be the most effective setting (see §6.3).

4.3 Active Set Update

Another challenge remaining in our framework is to effectively update the active set $C^{[k]}$ to ensure large step size $\alpha^{[k]}$ while restricting the number of active constraints. When advancing $\mathbf{x}^{[k]}$ to $\hat{\mathbf{x}}^{[k+1]}$ using CCD, if the primitive pairs previously blocking the CCD from $\mathbf{x}^{[k-1]}$ to $\hat{\mathbf{x}}^{[k]}$ are not resolved, they will persist producing near-zero collision times in the new CCD queries, thereby resulting in tiny $\alpha^{[k+1]}$. Therefore, a natural way to update the active set is directly adding all primitive pairs generating intersections during the last CCD into $C^{[k+1]}$.

This simple strategy proves ineffective in scenarios involving large deformations and extremely complex contacts. Large variations between adjacent timesteps caused by large h or velocity often leads to severe penetration in the intermediate states $\hat{\mathbf{x}}^{[1]}, \hat{\mathbf{x}}^{[2]}, \dots$ during the early iterations. Under severe penetration, one primitive may pass through multiple layers of surfaces during the CCD, thereby generating far more active constraints than the actual ones. These unnecessary constraints may significantly interfere with

Algorithm 3 UpdateActiveSet

Input: intersection-free state $\mathbf{x}$, target state $\hat{\mathbf{x}}$, current active set $C = \{(i, \lambda_i, \gamma_i)\}_{i \in \mathcal{I}_C}$.

Output: updated active set C' .

```

1:  $C' = C$ ;
2: Collect all primitive pairs  $\mathcal{I}'$  that generate intersections during transition from  $\mathbf{x}$  to  $\hat{\mathbf{x}}$ ; ▷ BVH & CCD queries
3:  $\mathcal{I}' \leftarrow \mathcal{I}' \setminus \mathcal{I}_C$ ;
4: for all  $i \in \mathcal{I}'$ ,  $T_i \leftarrow$  CCD collision time of primitive pair  $i$ ;
5: for all vertex  $v$  do
6:    $T_v \leftarrow \min_{i \in \mathcal{I}' \text{ and } v \in i} T_i$ ;
7: end for
8: for all  $i \in \mathcal{I}'$  do
9:   if  $T_i \in \{T_v \mid v \in i\}$  then
10:     $\lambda_i, \gamma_i \leftarrow 0, 1$ ;
11:     $C' \leftarrow C' \cup \{(i, \lambda_i, \gamma_i)\}$ ;
12:   end if
13: end for
14: for all  $(i, \lambda_i, \gamma_i) \in C'$  do
15:   if  $\gamma_i < 0.01$  then
16:     $C' \leftarrow C' \setminus \{(i, \lambda_i, \gamma_i)\}$ ;
17:   end if
18: end for
19: return  $C'$ ;

```

the optimization process, leading to conflicting constraints and increased computational cost.²³⁴

We aim to address this issue by filtering the constraints generated by the simple update strategy. For each vertex v , we first find the primitive pair with the earliest collision time among the newly generated constraints containing vertex v . We then keep only those primitive pairs corresponding to the earliest collision pair of at least one vertex. This filtering strategy does not lead to stagnation caused by missing essential constraints, as any primitive pair consistently detected by CCD will eventually pass the filtering and enter the active set. As shown in §6.3, the filtering significantly reduces the number of active constraints, while effectively contributing to advancing the intersection-free trajectory. Algorithm 3 details the subroutine for our active set update. The newly generated constraints are assigned $\lambda_i = 0$ and $\gamma_i = 1$. As stated in §4.2, decay factors of inactive constraints will be progressively reduced, and these constraints will be removed once the decay factor falls below the threshold value 0.01.

4.4 Termination

In §4.1, we described the TOI-based termination criterion (Equation 9) employed by our method. We now justify that using a sufficiently small threshold ϵ and setting the minimum iteration count $K_{\min} = 1$ offers a minimal yet effective condition for achieving reasonably accurate simulation results. Furthermore, we show that, under linear elasticity, this criterion guarantees finite-step termination and first-order accurate time integration of our method.

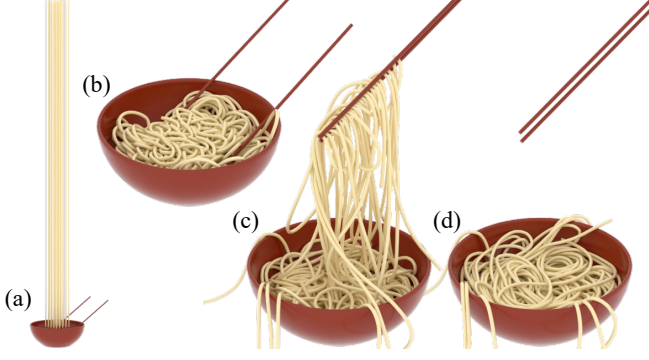


Fig. 5. **Ramen.** An array of long noodles is dropped into a fixed bowl, stably picked up using chopsticks with static friction ($\mu_f = 0.1$), and then released.

Preserving Motion in Non-Contacting Regions. Using a sufficiently small ϵ and setting $K_{\min} = 1$ ensures that the motion of non-contacting regions is not inadvertently constrained by time-of-impact (TOI) clamping in contact-rich areas. Consider a system with a contact-rich region and an isolated particle moving at constant velocity $\mathbf{v}$. Upon termination, the particle’s net displacement over the timestep is $(1 - \beta_0^{[k]})h\mathbf{v}$, which depends on the step sizes $\alpha^{[1]}, \dots, \alpha^{[k]}$ given by CCD in the contact region. By enforcing $\beta_0^{[k]} < \epsilon$ in the termination criterion, the particle retains at least $(1 - \epsilon)$ of its intended velocity, thereby avoiding artificially damped motion. A similar argument holds for non-contacting regions of elastic solids, where the resulting motion is at the same level of accuracy as that produced by semi-implicit Euler time integration.

Termination Guarantee. To satisfy the TOI-based termination criterion in Equation 9, our method must avoid stagnation at $\alpha^{[k]} = 0$. When $\alpha^{[k]}$ remains strictly positive, each $\beta_i^{[k]}$ will become a product of values in $[0, 1)$ and converge to zero, ensuring eventual termination. The generation of nonzero $\alpha^{[k]}$ is guaranteed by the combination of our active set update and augmented Lagrangian framework. If $\alpha^{[k]}$ temporarily becomes zero, both $\mathbf{x}^{[k]}$ and the linearized constraints $\mathbf{c}(\hat{\mathbf{x}}^{[k]})$ remain unchanged during the outer iteration. In this case, Algorithm 3 adds all primitive pairs immediately responsible for the zero $\alpha^{[k]}$ into the active set $\mathcal{C}^{[k]}$ for subsequent iterations. According to Theorem 17.6 of Nocedal and Wright [2006], when the elasticity model is linear and the penalty stiffness μ is sufficiently large, the errors of both the primal and dual variables relative to the true KKT solution of Equation 5 decay exponentially. Consequently, once the newly added contact pairs are included, the corresponding linearized constraints $c_i(\hat{\mathbf{x}}^{[k+1]}) = d_i(\mathbf{x}^{[k]}) + \nabla d_i(\mathbf{x}^{[k]})^T(\hat{\mathbf{x}}^{[k+1]} - \mathbf{x}^{[k]}) - \delta$ converge toward zero during the next subproblem solve. With $\delta > 0$, the resulting update direction $(\hat{\mathbf{x}}^{[k+1]} - \mathbf{x}^{[k]})$ eventually aligns with the positive gradient directions of all $d_i(\mathbf{x}^{[k]})$, thereby producing a strictly positive $\alpha^{[k+1]}$ in the next CCD.

First-order Accuracy. We first examine the case where no contact exists in the scene, in which our solver is simply equivalent to performing $K_{\min}$ Newton iterations. Let $\mathbf{v}^{[k]} = (\mathbf{x}^{[k]} - \mathbf{x}^t)/h$, when we set $K_{\min} = 1$ (also known as semi-implicit Euler in some literature),

$$\begin{aligned} \mathbf{v}^{[k+1]} &= (\mathbf{x}^{[k]} - \frac{M(\mathbf{x}^{[k]} - \mathbf{x}^t - h\mathbf{v}^t) + h^2\nabla U(\mathbf{x}^{[k]})}{M + h^2\nabla^2 U(\mathbf{x}^{[k]})} - \mathbf{x}^t)/h \\ &= \mathbf{v}^{[k]} - \frac{M(\mathbf{v}^{[k]} - \mathbf{v}^t) + h\nabla U(\mathbf{x}^{[k]})}{M + O(h^2)} \\ &= \mathbf{v}^t - hM^{-1}\nabla U(\mathbf{x}^t + h\mathbf{v}^{[k]}) + O(h^2) \quad \text{by Neumann series} \\ &= \mathbf{v}^t - hM^{-1}\nabla U(\mathbf{x}^t) + O(h^2), \end{aligned} \tag{15}$$

where we denote $A/B = B^{-1}A$ for matrices A and B , and assume Lipschitz continuity of the internal force ∇U . Since, in the 2nd term here, $-M^{-1}\nabla U(\mathbf{x}^t)$ is the time derivative of $\mathbf{v}^t$, we know that the local truncation error of $\mathbf{v}^{[k+1]}$ (and similarly for $\mathbf{x}^{[k+1]}$) is $O(h^2)$ for any finite k .

When contact is present, we first consider time intervals during which the active constraint set remains unchanged. With sufficiently large μ and small δ , our solver is equivalent to performing semi-implicit Euler (Equation 15) within the linear subspace satisfying all active constraints, and thus retains an $O(h^2)$ local truncation error. When the active constraint set changes, they happen momentarily, and assuming a smooth object trajectory with $h \rightarrow 0$, there are only a finite number of such moments. Thus, even if our method needs several time steps to fully include all active constraints and remove inactive ones, the accumulated error is $O(1)O(h)$, since each such time step would introduce an $O(h)$ contact force error. Combined with the accumulated per time step error analyzed in Equation 15, the total accumulated error becomes $O(1)O(h) + O(h^{-1})O(h^2) = O(h)$, i.e., our method is first-order accurate as a time integrator. We further demonstrate this first-order accuracy experimentally in §6.1 (see Figure 7) by evaluating our method on a 1D contact problem with an analytical solution.

For nonlinear elasticities, the above discussions still apply when we linearize the elasticity at each timestep, or iteratively optimizing $(\hat{\mathbf{x}}, \mathbf{s})$ by solving a sequence of approximated quadratic programs until they converge. In practice, we observed that Algorithm 2 with line search is sufficient to robustly and accurately handle the nonlinear and non-invertible elasticities by directly solving the nonlinear programming problem.

In §6.3, we demonstrate that the TOI-based termination criterion can also be applied to IPC, improving its performance without introducing damping artifacts. Nonetheless, our method remains significantly faster overall, owing to a higher average TOI and improved conditioning.

5 SIMULATOR DESIGN AND OPTIMIZATION

5.1 GPU Optimization

All core components of our simulator, including collision detection, Hessian assembly, linear system solving, and state updates, are GPU-parallelized through CUDA kernels. Our implementation utilizes Thrust and cuBLAS for efficient GPU data management and linear algebra operations. We store the symmetric Hessian matrix in a sparse 3×3 Block Sparse Row (BSR) format, where only the diagonal and upper-triangular blocks are assembled. The linear systems are solved using the conjugate gradient method with a 3×3 block-Jacobi

preconditioner, where the symmetric BSR SpMV is accelerated via warp-level reduction as in Huang et al. [2025a], which significantly reduces writing conflicts caused by atomic operations. The collision detection is accelerated using Linear BVH [Karras 2012] in the broad phase and filtered with ACCD [Li et al. 2021] in the narrow phase.

Conflict-Free Accumulation of Analytic PSD Elasticity Hessians. In addition to collision detection and linear system solving, another time-consuming part of elastodynamic simulators on the GPU lies in the positive semidefinite (PSD) projection and assembly of the Hessian matrix, during which the local Hessian matrices of tetrahedral elements are projected onto the PSD region, and then assembled into the global Hessian in BSR format. Computing PSD projection of the local elasticity Hessian can easily become a bottleneck if iterative algorithms are used for numerical eigendecomposition. A typical way to speedup this process is to analytically compute the eigenvalues λ_k and the corresponding eigenvectors $\text{vec}(\mathbf{Q}_k)$ of the 9×9 Hessian with respect to the deformation gradient $\mathbf{F}$. For isotropic elastic energies, these eigenvectors take the form

$$\mathbf{Q}_k = \mathbf{U} \mathbf{D}_k(\Sigma) \mathbf{V}^T, \quad k = 0, 1, \dots, 8, \quad (16)$$

where $\mathbf{F} = \mathbf{U} \Sigma \mathbf{V}^T$ is the SVD of $\mathbf{F}$ [Kim and Eberle 2020; Smith et al. 2018]. The matrices $\mathbf{D}_k(\Sigma)$ are diagonal for $0 \leq k \leq 2$ and antisymmetric with only two nonzero entries for $3 \leq k \leq 8$ (the $1/\sqrt{2}$ factor in some literature is absorbed into $\mathbf{D}_k$). A typical implementation (as in GIPC [Huang et al. 2024]) of this analytical projection directly assembles the projected 9×9 Hessian

$$\left(\frac{\partial^2 \Psi}{\partial \mathbf{F}^2} \right)^+_{i,j} = \sum_{k=0}^8 \lambda_k^+ \text{vec}(\mathbf{Q}_k) \text{vec}(\mathbf{Q}_k)^T, \quad \lambda_k^+ = \max(\lambda_k, 0) \quad (17)$$

and then transforms it back to the 12×12 form via two dense matrix multiplications. However, since the local assembly is parallelized at the tetrahedral element level, atomic operations are required when adding local matrices to the global Hessian, where we observe severe write conflicts that significantly limit performance. GIPC avoids write-conflict overhead by storing the Hessian in a matrix-free format that duplicates overlapping vertex blocks; however, this design increases the overhead of the subsequent SpMV operations in the PCG solve, which typically tends to be the more significant bottleneck in the simulation.

We propose a novel approach for improved GPU parallelization of the projection, which is parallelized at the vertex-block level to avoid write-conflict overhead via warp-level reduction, while also significantly reducing computation by exploiting the sparsity of $\mathbf{D}_k$. We first consider a 3×3 block in the previous 9×9 Hessian:

$$\begin{aligned} \left(\frac{\partial^2 \Psi}{\partial \mathbf{F}^2} \right)^+_{i,j} &= \sum_{k=0}^8 \lambda_k^+ \text{vec}(\mathbf{Q}_k)_{3i:3i+3} \text{vec}(\mathbf{Q}_k)_{3j:3j+3}^T \\ &= \sum_{k=0}^8 \lambda_k^+ (\mathbf{U} \mathbf{D}_k \mathbf{V}_i^T) (\mathbf{U} \mathbf{D}_k \mathbf{V}_j^T)^T \\ &= \mathbf{U} \left[\sum_{k=0}^8 \lambda_k^+ \mathbf{D}_k (\mathbf{V}_i^T \mathbf{V}_j) \mathbf{D}_k^T \right] \mathbf{U}^T, \end{aligned} \quad (18)$$

where $0 \leq i, j \leq 2$ are the block indices, and $\mathbf{V}_i, \mathbf{V}_j$ denote the corresponding rows of $\mathbf{V}$. When transforming back to 3×3 block

with respect to vertices $x_{i'}$ and $x_{j'}$, we sum up the ij -blocks by

$$\begin{aligned} \left(\frac{\partial^2 \Psi}{\partial x_{i'} \partial x_{j'}} \right)^+ &= \sum_{0 \leq i, j \leq 2} \mathbf{A}_{i',i} \mathbf{A}_{j',j} \left(\frac{\partial^2 \Psi}{\partial \mathbf{F}^2} \right)^+_{i,j} \\ &= \mathbf{U} \left[\sum_{k=0}^8 \lambda_k^+ \mathbf{D}_k (\mathbf{A}_{i'} \mathbf{V})^T (\mathbf{A}_{j'} \mathbf{V}) \mathbf{D}_k^T \right] \mathbf{U}^T, \end{aligned} \quad (19)$$

where $\mathbf{A}_{i'}$ (resp. $\mathbf{A}_{j'}$) is the constant row vector mapping $dx_{i'}$ to $d\mathbf{F}$ (in an outer product $d\mathbf{F} = dx_{i'} \mathbf{A}_{i'}$), which can be precomputed at the beginning of the simulation. In each thread, we process an $i'j'$ -block by first computing the outer product $(\mathbf{A}_{i'} \mathbf{V})^T (\mathbf{A}_{j'} \mathbf{V})$, then expanding the multiplications with $\mathbf{D}_k$ according to their sparse structures, and finally multiplying the result by $\mathbf{U}$ and $\mathbf{U}^T$. The threads are launched in the order of i' and j' , so that all Hessian blocks corresponding to the same pair (i', j') are aggregated via warp-level reduction, thereby alleviating atomic write conflicts. Compared to the direct analytical PSD projection, we achieve 2.34 $\times$ fewer multiplications and 3.17 $\times$ faster elasticity Hessian assembly. On our large-scale benchmark (Figure 4), the elasticity Hessian assembly still accounts for 11.6% of the total runtime after optimization, highlighting the importance of this improvement.

5.2 Conditioning-Aware Adjustment of μ

The theoretical termination guarantee of our method depends on sufficiently (though not infinitely) large μ and small δ in our augmented Lagrangian solver. An insufficiently small δ may cause conflicts among constraints, but fortunately, it can be set according to mesh resolution just like the distance threshold $\hat{d}$ in IPC. However, μ is not so intuitive, and setting it too large can easily make the system ill-conditioned. Thus, at the beginning of each time step, we estimate an initial value of μ , setting it as large as possible while keeping the conditioning of $\nabla^2 \mathcal{L}$ on the same level as that of $\nabla^2 E$. In practice, we use the following approximation:

$$\mu_{\text{init}} = C \max_i (\nabla^2 E)_{ii}, \quad (20)$$

which works well with a constant value of $C = 0.1$ used through all test cases.

Theoretically, μ and δ should be adaptively adjusted when the iterations stall at zero TOI. We employ an adaptive mechanism that updates $\mu \leftarrow 2\mu$ and $\delta \leftarrow \delta/2$ when $\alpha^{[k]} < 10^{-4}$ for 50 consecutive iterations; however, this condition was never triggered in any of our experiments, as the estimated μ_{init} already works effectively in practice. For example, in our twisting rods test (Figure 6), μ_{init} increases from $\sim 10^1$ to $\sim 10^4$ as the deformation becomes increasingly severe.

5.3 Penalty-Free Moving Boundary Conditions

Another advantage of our method over barrier-based approaches is its ability to efficiently handle moving boundary conditions. In the IPC framework [Huang et al. 2024; Li et al. 2020a], since the updated target positions of moving boundaries may penetrate other geometries, an auxiliary spring energy is introduced to pull all moving boundary vertices toward their new positions, after which the moving boundary DoFs are eliminated from the system solve. This process requires additional iterations with poor conditioning due

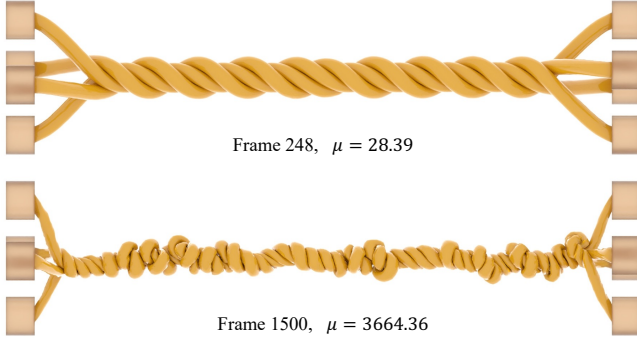


Fig. 6. **Twisting rods.** A bundle of elastic rods with boundaries rotated by 5400° in opposite directions at both ends. The contact stiffness μ is adaptively adjusted to match the conditioning of the Neo-Hookean elasticity.

to the large spring stiffness. Our method avoids these extra iterations, as the intermediate states $\hat{\mathbf{x}}^{[k]}$ allow penetration between the boundaries and interior geometries. In the initial state $\hat{\mathbf{x}}^{[0]}$, we move all moving boundary vertices to their target positions and eliminate the moving boundary DoFs by zeroing out all corresponding entries in the system gradient and Hessian, except for the diagonals, in subsequent linear solves. Since all $\hat{\mathbf{x}}^{[k]}$ satisfy the new boundary conditions, the boundaries in the final $\mathbf{x}^{[k]}$ is a linear blend between the target positions and the old boundaries in $\mathbf{x}^t$, with a blending weight on the target positions greater than $1 - \epsilon$. The upper bound of the moving boundary position error is then

$$e_{\text{boundary}} \leq \frac{\epsilon h v_{\text{boundary}}}{1 - \epsilon}, \quad (21)$$

where v_{boundary} denotes the maximum velocity of the moving boundary vertices.

6 EVALUATION

We conducted all evaluations on a desktop PC equipped with an Intel Core i9-13900K CPU (24 cores), 64 GB of RAM, and an NVIDIA GeForce RTX 4090 GPU. In all experiments, including the compared methods, we use the same CG relative error tolerance of 10^{-4} and a termination threshold of $\epsilon = 10^{-3}$ for all methods employing TOI-based termination. We set $K_{\min} = 6$ for all cases involving friction and cloth and use $K_{\min} = 2$ otherwise (except Twisting cloth, see below). We present the unit tests in §6.1, stress tests in §6.2, ablation studies in §6.3, and comparisons with state-of-the-art penetration-free simulators in §6.4.

6.1 Unit Tests

Convergence to analytic solution. We first demonstrate the first-order accuracy of our algorithm on two one-dimensional contact problems with known analytic solutions. Following Doyen et al. [2011]; Li et al. [2023b], we test the collision between a linearly elastic bar and a rigid ground under two scenarios: (1) with an initial velocity but no gravity, and (2) with gravity but no initial velocity. We first calculate the exact solution for a fixed spatial discretization, and then progressively refine the time step sizes. As shown

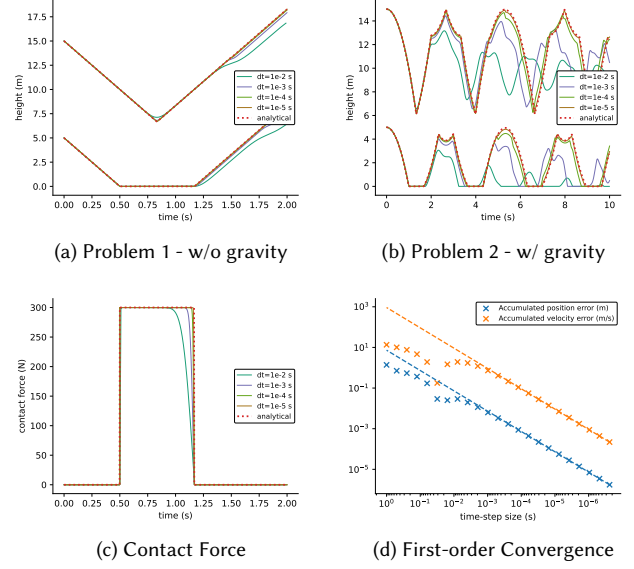


Fig. 7. **First-order convergence.** As the time-step size is refined, our simulation results (heights of the elastic bar’s endpoints) converge to the analytic solutions in both problems as shown in (a) and (b). We further show for problem 1: (c) the convergence of contact force and (d) first-order convergence of accumulated position and velocity errors (the dashed reference lines are with slope 1).

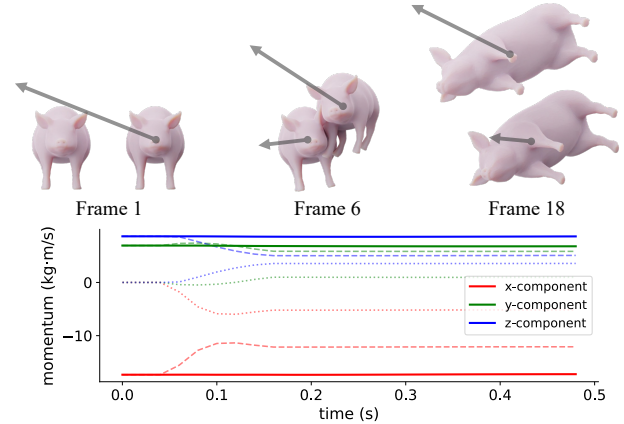


Fig. 8. **Momentum conservation.** The plot shows the evolution of total and individual momenta, where solid lines represent the total momentum and dotted lines denote each body.

in Figure 7a and 7b, as the temporal discretization is refined, the simulated trajectories eventually converge to the analytic solutions. Figure 7c further shows the convergence of contact force between the objects in problem 1. The order of convergence is demonstrated in Figure 7d, where the accumulated position and velocity errors in problem 1 are plotted against time step sizes on a log-scale, with the data points following the reference lines with slope 1.

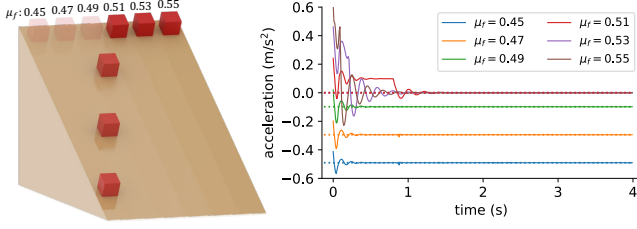


Fig. 9. **Cubes on a slope.** Cubes placed on a 30° inclined surface with friction coefficients ranging from 0.45 to 0.55, demonstrating accurate frictional behavior near the sliding threshold.

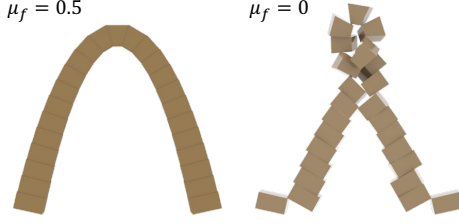


Fig. 10. **Masonry arch.** Following Li et al. [2020a], we demonstrate robust static friction with coefficient $\mu_f = 0.5$ that supports the arch structure.

Momentum conservation. We validate the conservation of system’s total linear momentum during frictional contact between two soft bodies, as shown in Figure 8. We apply an initial velocity on one of the body, making it collide with the other body and transfer its momentum through contact and friction forces. After the two bodies separate, the system reaches a state of constant motion while preserving the same total momentum as in the initial configuration.

Sliding friction. We evaluate the accuracy of our friction model by placing initially stationary cubes on a fixed surface inclined at 30° (Figure 9). The friction coefficients between the cubes and the surface range from 0.45 to 0.55, covering the threshold value of 0.5 that allows sliding. Our method accurately simulates the frictional behavior with friction coefficients near the threshold value, as shown in the acceleration curves versus analytic solutions.

Static friction. Following Li et al. [2020a], we include the masonry arch benchmark for static friction. We model the arch as a set of nearly rigid blocks with a Young’s modulus of $E = 10$ MPa, with the two bottom blocks fixed as boundary supports (Figure 10). We observe that the blocks form a stable arch structure under a friction coefficient of $\mu_f = 0.5$, while collapsing when $\mu_f = 0$ due to sliding between the blocks.

High-resolution mesh under large δ . A well-known issue [Chen et al. 2025; Huang et al. 2025b] of IPC is that when the mesh resolution is sufficiently high that the surface triangle sizes are smaller than $\hat{d}$ (corresponding to δ in our method), the barrier penalty generates contact forces between geodesically close triangle pairs, which can lead to severe artifacts and a large number of false contact pairs (see the second row of Figure 11). In comparison, our method only

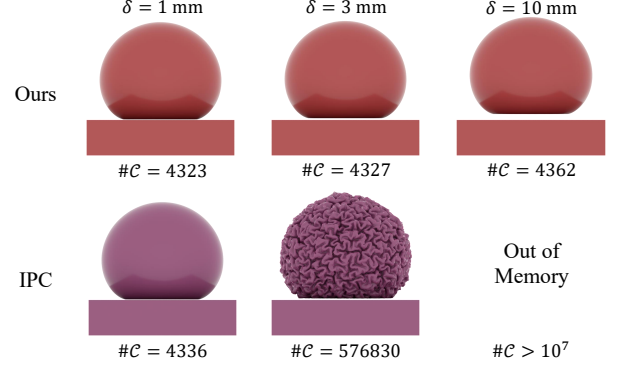


Fig. 11. **High-resolution mesh under large contact radius.** Dropping a high-resolution sphere with 50k vertices. The number of contact pairs stays consistent in our method, while IPC generates a large number of false contact pairs and lead to severe artifacts (bottom middle) or runs out of memory (bottom right).

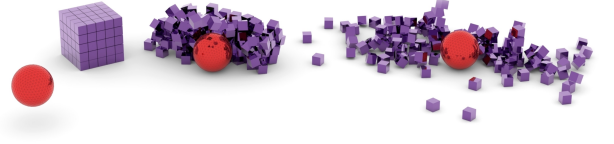


Fig. 12. **Near-rigid bodies.** A near-rigid sphere with $E = 1$ GPa collides with a pile of equally stiff cubes, generating intense collisions and rolling behavior induced by ground friction.

adds colliding primitive pairs detected during CCD into the constraint set, and thus avoids generating contact pairs that are in fact not colliding. As shown in Figure 11, with increasing δ , our method maintains a stable number of active constraints and does not suffer from the artifacts observed in IPC.

6.2 Stress Tests

Near-rigid bodies. We evaluate the ability of our algorithm to simulate contact of near-rigid objects with high stiffness. As illustrated in Figure 12, we simulate a moving sphere crashing into a pile of cubes, with all objects having a large Young’s modulus of $E = 1$ GPa. Our simulation remains stable and efficient during the high-speed impact and the complex contacts between the objects, while also accurately handling the sphere’s rolling behavior induced by ground friction (see our supplemental videos).

Extremely high-speed contact. We evaluate the robustness of our method using a challenging test case in which a soft body collides with a fixed thin plate at an extremely high speed of 100 m/s (Figure 13), a scenario that most methods lacking penetration-free guarantees will suffer from tunneling artifacts. During contact, the huge momentum results in an extreme compression of the body into a thin layer, after which it recovers its shape and bounces back. The whole process remains stable and penetration-free.

Squishy balls under extreme compression. We present a challenging test case to evaluate our method’s robustness under complex

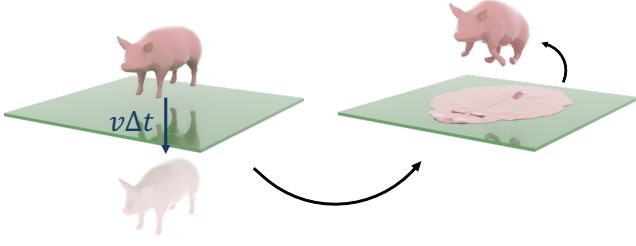


Fig. 13. **Pig falling.** A pig falls onto a fixed thin plate at an extremely high speed (100 m/s), compressed into a thin layer, and bounces back to recover its rest shape. Our results remain penetration-free throughout the process.

contacts and extreme stress. As shown in Figure 1, we simulate five elastic squishy balls compressed by a hydraulic press machine, modeled as a moving Dirichlet boundary. We first progressively shrink the internal space of the container, reaching a minimum height of 5 mm, which results in an extreme compression of the squishy balls, generating 1.45M contact pairs at peak. We then quickly release the top plate, allowing the squishy balls to recover their shapes and rebound as a result of the stored elastic potential energy.

Animal well. To evaluate the scalability of our method, we simulate a scene with a large collection of elastic animal toys falling into a fixed square well, as illustrated in Figure 4. The system contains 1.29M DoFs and 1.34M tetrahedral elements, which generates up to 181.4k contact pairs at peak. Under gravitational acceleration, the objects at the top reach a maximum speed of 19.7 m/s when colliding with the lower bodies, such that the distance traveled in a single time step exceeds the body size. This again illustrates the robustness of our method under high-speed impacts and complex contact in high-resolution scenarios.

Compressing chains. We introduce another challenging example to demonstrate our method’s robustness under extreme compression. As shown in Figure 3, three nested elastic chain rings with a Young’s modulus of $E = 100$ kPa are compressed within a shrinking box-shaped boundary. The volume occupied by the rings decreases from 0.04 m^3 to 215.9 cm^3 during compression, resulting in an average density increase of $185.2\times$. The boundary is then released, making the chains rapidly expand outward driven by the high elastic potential energy stored during compression. Our method robustly simulates the compression and expansion of the chains, preserving their exact topology even under extreme compression.

Ramen. As shown in Figure 5, the Ramen test demonstrates complex frictional contacts, in which 25 ramen noodles (12 meters long, Young’s modulus $E = 32$ kPa) are dropped into a fixed bowl and subsequently picked up with a pair of chopsticks. With a friction coefficient of $\mu_f = 0.1$ between the chopsticks and the noodles, we are able to simulate stable static friction that prevents the noodles from sliding down. The gripped noodles naturally slip off when the gap between the two chopsticks increases. The simulation is efficiently time stepped at 0.33s per time step ($h = 0.02\text{s}$).

Twisting rods. We follow Li et al. [2020a] to introduce the example of a bundle of twisting rods (Figure 6), each with a stiff Young’s

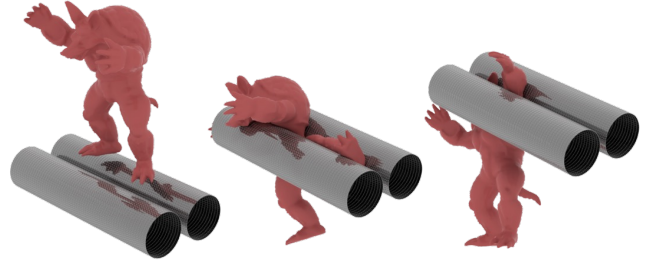


Fig. 14. **Friction roller.** A stiff Armadillo model ($E = 10$ MPa) is dropped onto two rotating rollers with friction coefficient $\mu_f = 0.1$, demonstrating robust frictional contact under large deformation.

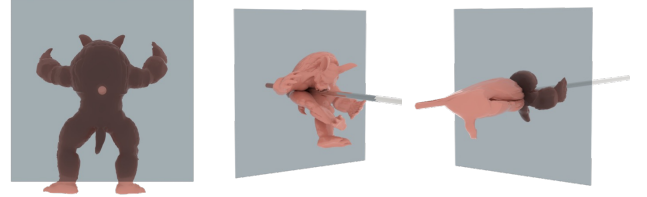


Fig. 15. **Armadillo through a tiny hole.** A stiff Armadillo model ($E = 10$ MPa, 10 m tall) is pushed through a narrow circular hole by a moving rod, demonstrating robust handling of extreme deformation and high stress.

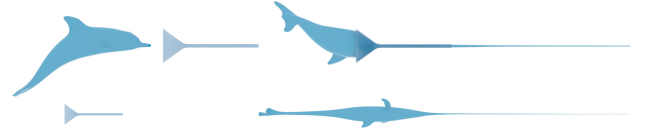


Fig. 16. **Dolphin & funnel.** The funnel test from Li et al. [2020a], showing a penetration-free trajectory of the dolphin passing through a narrow funnel.

modulus of $E = 1$ MPa. With both ends rotating at $180^\circ/\text{s}$ ($2.5\times$ faster than the original setting) in opposite directions for 30 s, we generate large deformations on the four thin elastic rods and observe strong buckling effects under high stress.

Friction roller. Another test case (Figure 14) from Li et al. [2020a] demonstrates the robustness of friction under large deformation. A stiff Armadillo model with Young’s modulus of $E = 10$ MPa is dropped onto a pair of fixed rollers rotating at a constant speed, with a friction coefficient of $\mu_f = 0.1$ applied between them. We robustly handle the static friction between the objects, which drives the Armadillo downward through the narrow gap against the large elastic resistance. The simulation proceeds efficiently with $h = 0.02\text{s}$, achieving an average runtime of 0.15s per step, even under large deformations and frictional contact.

Funnel. We also include the funnel test from [Li et al. 2020a], in which a stiff elastic dolphin model is dragged through a long, narrow funnel (Figure 16). The stiffness of the dolphin and the small funnel

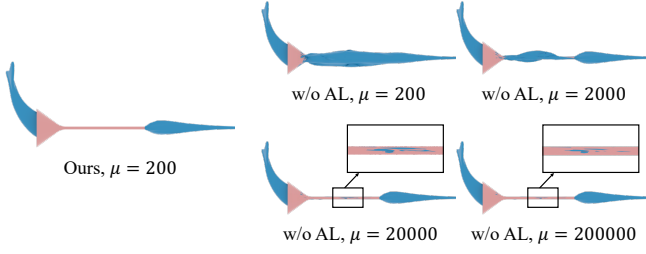


Fig. 17. **Augmented Lagrangian v.s. penalty method.** Without the Lagrange multiplier term, interpenetration cannot be resolved in $\hat{\mathbf{x}}^{[k]}$ even with $1000\times$ larger penalty stiffness.

size make it difficult to pass through the obstacle, requiring extreme elongation to generate sufficient elastic force. Our method achieves results similar to IPC, generating a penetration-free animation of the dolphin under extreme compression and elongation.

Armadillo Through a Tiny Hole. We present another challenging example involving extreme deformation and high internal stress: an stiff Armadillo model of Young’s modulus $E = 10$ MPa, approximately 10 m in its longest dimension, is driven through a narrow circular hole of radius 0.24 m in a fixed plate (Figure 15). The motion is induced by a slender rod of radius 0.18 m acting as a moving boundary. Similar to the funnel test, this scenario requires forcing a stiff elastic body of volume 25.7 m^3 through an aperture of area 0.086 m^2 , highlighting the robustness of our method under severe geometric and material constraints.

6.3 Ablation Study

Augmented Lagrangian v.s. penalty method. The efficiency and finite-step termination of our algorithm is enabled by the Augmented Lagrangian subproblem solver, which explicitly tracks the estimated Lagrange multipliers for the contact constraints and allows the use of a relatively small stiffness μ to keep the systems well-conditioned. To illustrate the effectiveness of our augmented IP formulation (Equation 12), we compare our full Augmented Lagrangian solver against using the simple quadratic penalty without the Lagrange multiplier term. Starting from a penetration-free state in the dolphin & funnel test case, we optimize $\hat{\mathbf{x}}^{[k]}$ using both our full augmented IP and the simple quadratic penalty. Both methods use the same offset $\delta = 1$ mm. As shown in Figure 17, the simple quadratic penalty generates severe penetrations in $\hat{\mathbf{x}}^{[k]}$ under $10\times$ larger stiffness μ , and noticeable penetrations still exist even under $1000\times$ larger μ . When constructing the penetration-free paths from $\{\hat{\mathbf{x}}^{[k]}\}_{k \geq 1}$, the penetrations in $\hat{\mathbf{x}}^{[k]}$ no longer decrease after the constraint set finalizes, causing the Newton iterations to stall at near-zero $\alpha^{[k]}$ and thus fail to converge.

Constraint filtering. As described in Section 3, the filtering process controls whether a newly detected contact is added to the constraint set $C^{[k]}$. When handling contact-rich scenes or fast-moving objects, the filtering mechanism significantly reduces the number of unnecessary constraints, as shown on the left of Figure 18. In extreme cases (e.g., after releasing the piston in the squishy balls compression test), without filtering, the excessive number of unnecessary

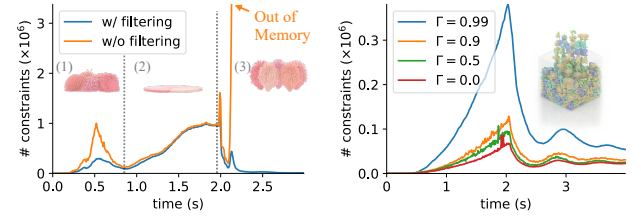


Fig. 18. **Constraint filtering and decay.** Evolution of contact constraint count w/ and w/o filtering in the squishy balls example (left) and with different constraint decay factors Γ in the animal well example (right).

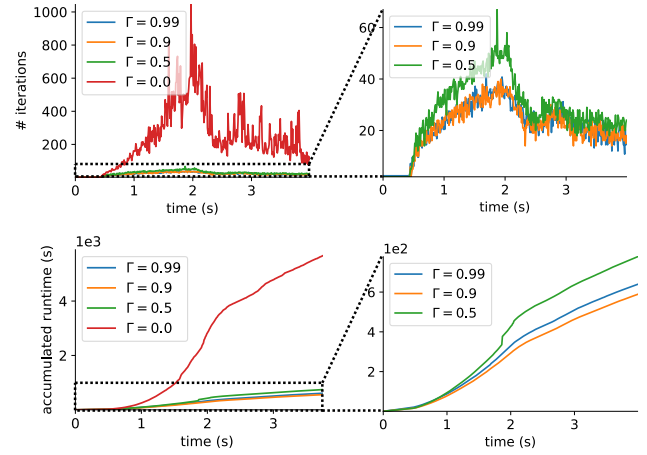


Fig. 19. **Effectiveness of constraint decay.** Evolution of Newton iteration count and accumulated runtime with different constraint decay factor Γ in the animal well example.

contacts could make the constraint set $10\times$ larger than ours, leading to higher computational cost and even out-of-memory issues.

Constraint decay. Just as filtering determines how $C^{[k]}$ is properly enlarged, the constraint decay mechanism governs how constraints that become inactive are smoothly removed from $C^{[k]}$. As stated in §4.2, we use the parameter Γ to control the rate at which the decay factors γ_i of inactive constraints gradually decreases, and remove them from $C^{[k]}$ once $\gamma_i < 0.01$. In the extreme case of $\Gamma = 1$, all added constraints remain permanently in $C^{[k]}$ across time steps, thus unnecessarily wasting resources after a majority of them become inactive. Figure 19 shows how varying values of $\Gamma < 1$ affect both the number of Newton iterations and the runtime required by the subproblem solver in the animal well test. When $\Gamma = 0$, all inactive constraints are instantly removed from $C^{[k]}$, causing the optimization objective to change drastically across iterations and potentially leading to constraints oscillating in and out of $C^{[k]}$. This makes it significantly harder for the subproblem solver to generate large $\alpha^{[k]}$, thereby requiring far more iterations and longer runtime. The values between $\Gamma = 0$ and $\Gamma = 1$ show a tradeoff between the number of iterations and the constraint set size, and we select $\Gamma = 0.9$ as the fixed value for our method due to its best performance. The

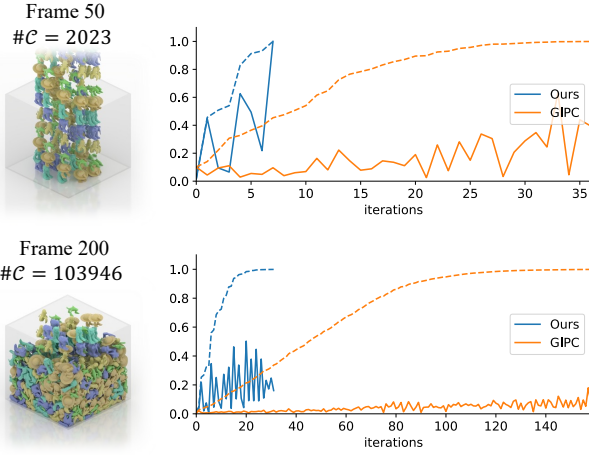


Fig. 20. **Progress of termination.** Solid line: per-iteration advanced TOI $\alpha^{[k]}$. Dashed line: accumulated TOI $(1 - \beta_0^{[k]})$, terminating upon reaching $1 - \epsilon$. Both our method and GIPC use the same contact radius $\delta = 1$ mm and termination threshold $\epsilon = 10^{-3}$, $K_{\min} = 1$.

relationship between Γ and the size of the constraint set is illustrated on the right of Figure 18.

Progress of Termination. A key advantage of our method over IPC lies in the TOI progress achieved per line search, resulting in substantially fewer Newton iterations under the TOI-based termination criterion. To quantify this improvement, we first applied IPC using the TOI-based termination criterion, which reduced the number of Newton iterations per time step by an average of $2.67\times$ in the animal well test, while maintaining comparable solution quality. To more clearly compare the progress made per Newton iteration, we compare our method and IPC (both using the TOI-based termination criterion) across scenarios with varying contact complexities. Figure 20 plots the trajectories of $\alpha^{[k]}$ and $\beta_0^{[k]}$ until the termination condition $\beta_0^{[k]} < \epsilon = 10^{-3}$ (with $K_{\min} = 1$) is met. Unlike IPC, which truncates the search direction at the earliest detected contact and discards subsequent information, our method tracks all newly detected contacts within the penetration-allowing state $\hat{\mathbf{x}}^{[k]}$ and the constraint set $C^{[k]}$. This enables significantly larger $\alpha^{[k]}$ values, particularly in contact-rich scenarios, allowing our solver to construct the penetration-free trajectory with an average of $4.24\times$ fewer Newton iterations.

6.4 Comparisons

GIPC [Huang et al. 2024]. Despite benefiting from substantial GPU acceleration, GIPC, as a representation of the barrier-based IPC pipeline, remains limited by the TOI locking issue and the ill-conditioning of the barrier function. We compare our method with GIPC on two of our large-scale cases (Figure 1 and Figure 4) and one smaller case (Figure 3), all involving high velocities, large deformations, and challenging contacts. We test GIPC under both the original residual-based termination criterion (with default $\epsilon_d = 10^{-2}l$) and the TOI-based termination criterion, using the same threshold of

$\epsilon = 10^{-3}$. As shown in Table 1, even with the TOI-based termination, GIPC requires $4.4\times$ – $10.3\times$ more Newton iterations than ours due to TOI locking. Under the same CG tolerance threshold, the ill-conditioning of the logarithmic barrier further leads to $3.79\times$ – $6.96\times$ more PCG iterations to converge. Considering all factors, including GPU optimization, our method achieves up to $103.15\times$ faster performance than GIPC with its original residual-based termination, and $59.18\times$ faster than GIPC with the same TOI-based termination as ours. In addition to the costs of Hessian construction and linear solves, GIPC suffers from a considerable overhead in line search as the BVH is reconstructed in every backtracking iteration. In contrast, our method naturally eliminates this expense as the constraint set is explicitly maintained and independent of the shape configuration during line searches.

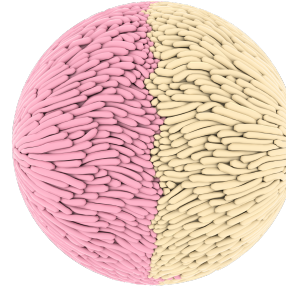


Fig. 21. **Trapped squishy balls.** Reproduced from the official repository of Cubic Barrier [Ando 2024].

and a test case of trapped squishy balls (Figure 21) from its official examples. For a fair comparison, we use the same TOI-based termination criterion with $\epsilon = 10^{-3}$ for both methods. Despite the performance disadvantage of double-precision arithmetic, we outperform the Cubic Barrier method (single-precision) in all cases, achieving up to $84.4\times$ faster performance. Their design of stiffness adjustment makes the contact Hessian to scale as $O(\mu d) = O(d^{-1})$, causing the system’s conditioning diverging as the gap distance d approaches zero, analogous to the IPC barrier. As shown by the statistics in Table 1, this not only increases the PCG cost in each linear solve but also leads to numerical instability issues, particularly when using single-precision arithmetic: In two of our challenging cases, Cubic Barrier fails due to PCG non-convergence near the most challenging part of the simulation. We also observe that Hessian assembly becomes the dominant cost when the number of contacts is large, primarily due to the inefficient GPU implementation of CSR matrix filling. Nonetheless, we still achieve a $23.9\times$ speedup when considering only the PCG phase in such scenarios.

Offset geometric contact [Chen et al. 2025]. Offset Geometric Contact (OGC) is a recent representative work that employs first-order iterative methods on a modified IPC barrier to achieve penetration-free simulation. Despite requiring more iterations to converge, the vertex-block descent scheme used in OGC avoids solving large linear systems and thus achieves faster per-iteration performance. Instead

Cubic Barrier [Ando 2024].

In another recent work, Ando [2024] proposed a cubic contact energy that semi-implicitly controls the contact stiffness across Newton iterations. To provide sufficient collision response, the contact stiffness of each individual contact pair is adjusted as $\kappa \leftarrow O(d^{-2} + \rho(H))$, according to the primitive distance d and an estimated spectral radius of the elasticity Hessian H . We compare our method with Cubic Barrier on two large-scale cases (Figure 1 and Figure 4)

	method	prec	#CG	#iters	#contacts	TOI	time (s)	accel	Hess	PCG	CCD	LS	misc
Figure 1	Ours	double	28.35	30.09	0.53M	0.192	5.367	-	0.762	1.229	3.124	0.134	0.118
	GIPC	double	129.03	408.93	0.67M	-	528.57	98.5×	34.51	305.46	33.65	154.66	0.161
	GIPC w/ TOI-term	double	107.58	137.42	0.62M	0.055	144.22	26.9×	12.72	76.59	15.07	39.78	0.062
	Cubic Barrier	single	145.24	33.13	1.15M	0.163	452.92	84.4×	414.42	29.44	8.605	-	0.453
Figure 3	Ours	double	67.10	16.13	42.6k	0.250	0.657	-	0.025	0.114	0.499	0.007	0.012
	GIPC	double	507.33	261.61	48.3k	-	67.77	103.15×	1.942	44.52	3.432	18.00	0.017
	GIPC w/ TOI-term	double	467.56	167.06	46.3k	0.046	38.88	59.18×	1.265	25.32	3.697	8.574	0.010
Figure 4	Ours	double	53.59	19.23	37.9k	0.242	1.300	-	0.237	0.534	0.446	0.051	0.033
	GIPC	double	142.90	228.79	47.3k	-	125.02	96.17×	16.11	77.11	10.43	20.73	0.558
	GIPC w/ TOI-term	double	215.34	85.58	45.9k	0.089	61.51	47.3×	6.231	43.41	3.950	7.673	0.152
	Cubic Barrier	single	433.38	61.04	60.6k	0.093	23.30	17.9×	6.571	14.56	2.097	-	0.070
Figure 21	Ours	double	44.65	11.80	27.0k	0.319	0.580	-	0.107	0.212	0.216	0.019	0.026
	Cubic Barrier	single	505.60	7.95	56.2k	0.441	2.930	5.05×	0.686	2.039	0.164	-	0.041
Figure 22 (easy)	Ours	double	24.90	2	529.67	0.833	0.0158	-	3.8e-3	3.2e-3	5.5e-3	1.6e-5	3.3e-3
	OGC ($N = 10$)	single	-	100	-	-	0.0184	1.2×	*	*	*	*	*
	OGC ($N = 50$)	single	-	2500	-	-	0.2382	15.1×	*	*	*	*	*
	OGC ($N = 80$)	single	-	6400	-	-	0.5225	33.1×	*	*	*	*	*
Figure 22 (hard)	Ours	double	51.47	2	1.8k	0.575	0.0216	-	3.3e-3	6.5e-3	7.3e-3	3.5e-4	4.2e-3
	OGC ($N = 10$)	single	-	100	-	-	0.0378	1.7×	*	*	*	*	*
	OGC ($N = 100$)	single	-	10k	-	-	1.7618	81.6×	*	*	*	*	*
	OGC ($N = 200$)	single	-	40k	-	-	5.8859	272.5×	*	*	*	*	*
Figure 23 (easy)	Ours	double	40.62	23.26	146.08k	0.229	1.598	-	0.364	0.312	0.839	0.043	0.040
	OGC ($N = 30$)	single	-	900	-	-	6.470	4.0×	*	*	*	*	*
	OGC ($N = 50$)	single	-	2500	-	-	16.65	10.4×	*	*	*	*	*
	OGC ($N = 80$)	single	-	6400	-	-	41.59	26.0×	*	*	*	*	*
Figure 23 (hard)	Ours	double	252.71	11.84	27.73k	0.366	1.683	-	0.146	0.808	0.667	0.019	0.044
	OGC ($N = 50$)	single	-	-	-	-	-	-	-	-	-	-	-

Yellow: shows noticeable artifacts or crashes near the end (see our supplementary video).

Red: crashes immediately after contacts occur.

* A detailed runtime breakdown of OGC is provided in the appendix.

Table 1. **Comparison configurations and statistics.** TOI-term stands for TOI-based termination. OGC uses N substeps and N iterations per substep. prec: precision of floating point arithmetic. #iters: average Newton or VBD iterations per time step. #contact: average number of contact pairs. TOI: average advanced TOI ($\alpha^{[k]}$) for methods using TOI-based termination. time: average runtime per time step. accel: speedup factor of our method over the compared method. Hess, PCG, CCD, LS, misc: runtime breakdown of Newton-type methods into Hessian computation, linear solve, CCD, line search, and others.

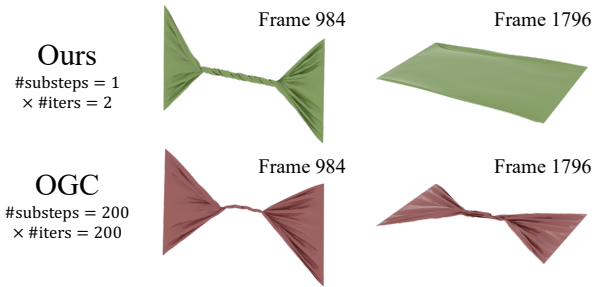


Fig. 22. **Comparison to OGC – Twisting cloth (hard version).** Both OGC [Chen et al. 2025] and our method uses the same contact radius $\delta = 2$ mm. OGC fails to recover the rest shape after twisting back even with 200 substeps and 200 iterations per substep, making it 2-orders-of-magnitude slower than our method with 2 Newton iterations per time step, which produces stable and realistic results.

of relying on a termination criterion, OGC provides direct control over the number of iterations per time step, offering a trade-off

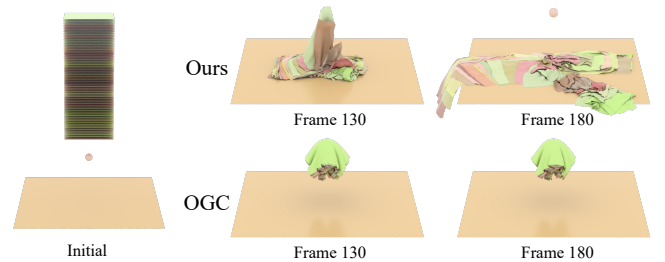


Fig. 23. **Comparison to OGC – Stacked cloth (easy version).** OGC uses 30 substeps and 30 iterations per time step, performing 4× slower than our method and suffering from locking artifacts with no friction. See our supplementary video for more visual comparisons.

between speed and simulation quality. Under challenging scenarios involving intense collisions or high velocities, we show that OGC may require a very large number of substeps and iterations to achieve visually plausible results, making it less competitive than Newton-based methods with optimized linear system solvers. In

	#V, #F, #T	h (s)	δ (m)	model	ρ (kg/m ³), E (Pa), ν	μ_f, ϵ_v (m/s)	$K_{\min}, \epsilon$	#CG	#Newton	#contact	time (s)
Fig 8	20.0k, 17.5k, 87.2k	0.02	1e-3	SNH	1e3, 1e6, 0.3	0.1, 1e-3	6, 1e-3	33.6 (619)	6.0 (6)	35.8 (803)	0.16 (0.17)
Fig 9	3.1k, 5.8k, 8.3k	0.01	1e-3	SNH	1e3, 1e6, 0.4	0.5±0.05, 1e-5	6, 1e-3	30.4 (47)	6.0 (6)	434.7 (600)	0.038 (0.043)
Fig 10	200, 300, 150	0.002	1e-6	SNH	1e3, 1e8, 0.3	0.5, 1e-6	6, 1e-3	303.1 (600)	28.4 (102)	154.5 (246)	0.80 (3.52)
Fig 11	49.1k, 81.9k, 160.2k	0.01	1e-2	SNH	1e3, 1e4, 0.3	-	2, 1e-3	40.6 (50)	2.0 (2)	4.3k (8.2k)	0.15 (0.17)
Fig 1	0.87M, 1.59M, 2.25M	0.01	1e-3	COR	1e2, 1e4, 0.4	-	2, 1e-3	28.3 (146)	30.1 (47)	0.53M (1.45M)	5.37 (12.39)
Fig 3	51.0k, 87.2k, 167.4k	0.02	2e-4	COR	1e3, 1e5, 0.3	-	2, 1e-3	67.1 (773)	16.1 (103)	42.6k (542.8k)	0.66 (56.86)
Fig 4	0.43M, 0.80M, 1.34M	0.01	1e-3	SNH	1e3, 5e5, 0.3	-	2, 1e-3	53.6 (669)	19.2 (37)	37.9k (102.6k)	1.30 (4.11)
Fig 5	37.4k, 67.6k, 95.8k	0.02	1e-3	SNH	1e2, 3.2e4, 0.3	0.1, 1e-3	6, 1e-3	73.0 (179)	20.8 (51)	5.4k (26.6k)	0.33 (0.97)
Fig 6	10.5k, 17.8k, 33.6k	0.02	2e-4	NH	1e3, 1e6, 0.3	-	2, 1e-3	82.5 (1520)	13.4 (31)	4.7k (6.7k)	0.18 (0.45)
Fig 12	1.9k, k, 4.0k	0.02	1e-3	SNH	1e4, 1e9, 0.3	0.1, 1e-3	6, 1e-3	200.7 (1168)	7.1 (27)	453.3 (2.7k)	0.13 (0.74)
Fig 13	10.0k, 8.8k, 43.6k	0.02	1e-3	COR	1e3, 1e5, 0.49	0.5, 1e-3	6, 1e-3	83.7 (280)	7.6 (195)	8.9k (31.3k)	0.17 (7.60)
Fig 14	21.2k, 32.9k, 55.0k	0.02	1e-3	SNH	1e3, 1e7, 0.3	0.5, 1e-3	6, 1e-3	143.0 (719)	9.4 (29)	1.2k (5.1k)	0.15 (0.45)
Fig 15	21.7k, 40.2k, 64.4k	0.02	1e-3	SNH	1e2, 1e7, 0.3	-	2, 1e-3	171.2 (3.8k)	16.9 (94)	5.8k (22.9k)	0.44 (5.83)
Fig 16	6.7k, 13.2k, 20.6k	0.02	1e-3	SNH	1e3, 1e5, 0.3	-	2, 1e-3	71.5 (1134)	8.1 (36)	1.3k (9.4k)	0.11 (1.58)
Fig 21	0.35M, 0.64M, 0.90M	0.01	1e-3	SNH	1e3, 5e5, 0.35	-	2, 1e-3	44.7 (238)	11.8 (34)	27.0k (187.4k)	0.58 (3.57)
Fig 22 [†]	2.5k, 4.8k, 0	1/60	2e-3	Cloth ($\mu_{\text{mem}}=1e3, k_{\text{bend}}=1e-3$)	-	-	2, 0.99*	24.9 (64)	2.0 (2)	529.7 (1.5k)	0.016 (0.026)
Fig 22 [‡]	2.5k, 4.8k, 0	1/60	2e-3	Cloth ($\mu_{\text{mem}}=1e3, k_{\text{bend}}=1e-3$)	-	-	2, 0.99*	51.5 (217)	2.0 (2)	1.8k (4.1k)	0.022 (0.041)
Fig 23 [†]	213.8k, 414.7k, 0	0.02	2e-3	Cloth ($\mu_{\text{mem}}=300, k_{\text{bend}}=0$)	-	-	6, 1e-3	40.6 (173)	23.3 (83)	146.1k (567.5k)	1.60 (9.39)
Fig 23 [‡]	213.8k, 414.7k, 0	0.02	2e-3	Cloth ($\mu_{\text{mem}}=300, k_{\text{bend}}=5e3$)	-	-	6, 1e-3	252.7 (1670)	11.8 (43)	27.7k (772.6k)	1.68 (14.08)
Fig 24	889, 1.3k, 3.3k	0.01	1e-3	NH	1e2, 1e5, 0.3	-	2, 1e-3	63.1 (202)	2.1 (8)	282.0 (676)	0.028 (0.122)

[†] Easy version. [‡] Hard version. * Forced to 2 iterations.

Table 2. **Experiment configurations and statistics.** #V, #F, #T: number of vertices, faces, tetrahedra. h : time step size. δ : contact offset/radius. model: stable Neo-Hookean (SNH) [Smith et al. 2018], Neo-Hookean (NH), corotated linear (COR) and cloth. ρ, E, ν : density, Young’s modulus, and Poisson’s ratio. μ_f, ϵ_v : friction parameters. $K_{\min}, \epsilon$: termination parameters. #CG: average (peak) CG iterations per linear solve. #Newton: average (peak) Newton iterations per time step. #contact: average (peak) number of contacts pairs. time: average (peak) runtime per time step.

addition to the sublinear convergence of coordinate descent methods, this is mainly due to the conservative bound b_v used to ensure penetration-free advancement, which can become extremely small in regions with intense collisions or high mesh resolution. As a result, the motion in collision-intensive regions may be severely locked, which is especially noticeable under high velocities.

Since the official GPU implementation of OGC currently supports only co-dimensional materials, we compare it with our method on two test cases involving complex cloth self-collisions, each with two levels of difficulty. We use the parameter N to control the speed-accuracy tradeoff in OGC, dividing each time step into N substeps, with each substep performing N iterations. The first test case of a twisting cloth (Figure 22) is taken directly from the examples of OGC, with all parameters kept unchanged except for the twisting duration (6 s in the easy setting / 30 s in the hard setting) and the addition of periodic back-and-forth twisting motions. Under the easy setting, OGC fails to recover the rest shape when the twisting angle returns to zero using $N = 10$ and 50, while succeeding using $N = 80$. In the harder case, OGC fails to recover the shape using all $N = 10, 100, 200$. In this test, we demonstrate our method’s controllability on the efficiency-accuracy trade-off while guaranteeing high-quality output. Specifically, we set our $K_{\min} = 2$ and use a large ϵ so that our method always run 2 Newton iterations per time step. As shown in Table 1, this makes our method achieve comparable performance to OGC with $N = 10$ and 33× faster than OGC with $N = 80$, while avoiding the mentioned artifacts.

Another test case (Figure 23) involves dropping 50 layers of cloth onto a fixed sphere, either with no bending energy (easy) or with a high bending stiffness (hard). In the easy case, we outperform OGC using $N = 30, 50, 80$, each suffering from different degrees of locking (see the supplementary video). In the harder case, OGC with $N = 50$ crashes immediately after contacts occur, primarily due to the use of single-precision floating point and the lack of energy line search.

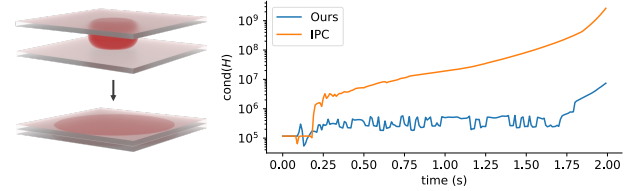


Fig. 24. **System conditioning.** Unlike the IPC barrier, our contact energy remains well-conditioned under high stress until the system conditioning is dominated by the Neo-Hookean elasticity at extreme compression.

Comparing system conditioning with IPC. Another advantage of our method compared to IPC is the better conditioning of linear systems, which significantly improved our performance. When using iterative linear solvers (e.g., PCG) in IPC, the log-barrier function becomes sharp at small distances, resulting in ill-conditioned linear systems that require specialized preconditioners [Huang et al. 2025a] or a large number of CG iterations to solve. Our method avoids this conditioning issue by employing a quadratic contact penalty (Equation 12), whose Hessian for each contact pair is always congruent to a constant matrix proportional to the contact stiffness μ . Taking advantage of the augmented Lagrangian method, a relatively small μ can be set to still yield accurate estimates of the optimal solution. We compare the system conditioning of our method and IPC in a simple sphere compression test, with both methods using their default estimated contact stiffness. As shown in Figure 24, our method offers a 2-orders-of-magnitude smaller system condition number in a compressing test, where even the simple block-Jacobi preconditioned CG can solve the systems in an average of 63.1 iterations.

7 CONCLUSION

We presented a novel barrier-free optimization framework for non-penetration elastodynamic simulation that combines an augmented Lagrangian formulation with efficient active-set exploration. In contrast to IPC-style approaches that rely on logarithmic barrier functions, our method models contact using a smooth augmented Lagrangian energy that remains well-conditioned under high stress and is theoretically guaranteed to provide sufficient collision response for consistent simulation progress. The proposed augmented Lagrangian solver, together with our novel constraint set update strategy, maintains a compact, anticipatory, and smoothly evolving constraint set, enabling rapid optimization progress without the TOI locking issues observed in IPC. Extensive experiments demonstrate substantial performance improvements and robustness across a wide range of challenging, contact-rich scenarios. To the best of our knowledge, this is the fastest existing method to achieve such a high level of robustness while maintaining accuracy. We believe it opens new possibilities for efficient, accurate, and reliable penetration-free simulation in time-sensitive applications such as robotics and virtual reality.

Although our method demonstrates strong efficiency and scalability, there remains significant potential for further optimization in certain components. As shown in our experiments, under collision-intensive scenarios, the cost of CCD surpasses the linear solver and becomes the dominating factor (2.69× slower than PCG in the stacked cloth test). The majority of the CCD cost arises in the broad phase during BVH queries, which currently do not exploit information from the active constraint set and therefore spend considerable time on unnecessary computations. Thus, a promising direction for further performance improvement is to design a more efficient CCD scheme optimized for our constraint set expansion. Another potential avenue for improvement lies in cloth simulation, which is currently supported by our framework but has not yet been fully optimized. In particular, developing specialized acceleration structures and constraint filtering strategies for co-dimensional geometries could further enhance both efficiency and robustness in large-scale contact-rich cloth and rod dynamics.

Acknowledgments

Tips: Keep formal. Include funding sources and major collaborative support from friends, colleagues, etc.

References

- Ryoichi Ando. 2024. A cubic barrier with elasticity-inclusive dynamic stiffness. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–13.
- Francisco Armero and Eva Petőcz. 1998. Formulation and analysis of conserving algorithms for frictionless dynamic contact/impact problems. *Computer methods in applied mechanics and engineering* 158, 3-4 (1998), 269–300.
- David J Benson and John O Hallquist. 1990. A single surface contact algorithm for the post-buckling analysis of shell structures. *Computer methods in applied mechanics and engineering* 78, 2 (1990), 141–163.
- Robert Bridson, Ronald Fedkiw, and John Anderson. 2002. Robust treatment of collisions, contact and friction for cloth animation. In *Proceedings of the 29th annual conference on Computer graphics and interactive techniques*. 594–603.
- Anka He Chen, Jerry Hsu, Ziheng Liu, Miles Macklin, Yin Yang, and Cem Yuksel. 2025. Offset Geometric Contact. *ACM Transactions on Graphics (TOG)* 44, 4 (2025), 1–21.
- Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. 2024. Vertex block descent. *ACM Transactions on Graphics (TOG)* 43, 4 (2024), 1–16.
- Yunuo Chen, Minchen Li, Lei Lan, Hao Su, Yin Yang, and Chenfanfu Jiang. 2022. A unified newton barrier method for multibody dynamics. *ACM Transactions on Graphics (TOG)* 41, 4 (2022), 1–14.
- Gilles Daviet, Florence Bertails-Descoubes, and Laurence Boissieux. 2011. A hybrid iterative solver for robustly capturing coulomb friction in hair dynamics. In *Proceedings of the 2011 SIGGRAPH Asia Conference*. 1–12.
- David Doyen, Alexandre Ern, and Serge Piperno. 2011. Time-integration schemes for the finite element dynamic Signorini problem. *SIAM Journal on Scientific Computing* 33, 1 (2011), 223–249.
- Wenxin Du, Siqiong Yao, Xinlei Wang, Yuhang Xu, Wenqiang Xu, and Cewu Lu. 2024. Intersection-free robot manipulation with soft-rigid coupled incremental potential contact. *IEEE Robotics and Automation Letters* 9, 5 (2024), 4487–4494.
- Wenxin Du, Chang Yu, Siyu Ma, Ying Jiang, Zeshun Zong, Yin Yang, Joseph Masterjohn, Alejandro Castro, Xuchen Han, and Chenfanfu Jiang. 2025. Embedded ipc: Fast and intersection-free simulation in reduced subspace for robot manipulation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 6911–6918.
- Yu Fang, Minchen Li, Chenfanfu Jiang, and Danny M Kaufman. 2021. Guaranteed globally injective 3D deformation processing. *ACM Transactions on Graphics* 40, 4 (2021).
- Zachary Ferguson, Pranav Jain, Denis Zorin, Teseo Schneider, and Daniele Panozzo. 2023. High-order incremental potential contact for elastodynamic simulation on curved meshes. In *ACM SIGGRAPH 2023 conference proceedings*. 1–11.
- Zachary Ferguson, Minchen Li, Teseo Schneider, Francisca Gil-Ureta, Timothy Langlois, Chenfanfu Jiang, Denis Zorin, Danny M Kaufman, and Daniele Panozzo. 2021. Intersection-free rigid body dynamics. *ACM Transactions on Graphics* 40, 4 (2021).
- Dewen Guo, Minchen Li, Yin Yang, Sheng Li, and Guoping Wang. 2024. Barrier-Augmented Lagrangian for GPU-based Elastodynamic Contact. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–17.
- David Harmon, Etienne Vouga, Rasmus Tamstorf, and Eitan Grinspun. 2008. Robust treatment of simultaneous collisions. In *ACM SIGGRAPH 2008 papers*. 1–4.
- Kemeng Huang, Floyd M Chitalu, Huancheng Lin, and Taku Komura. 2024. GIPC: Fast and stable gauss-newton optimization of IPC barrier energy. *ACM Transactions on Graphics* 43, 2 (2024), 1–18.
- Kemeng Huang, Xinyu Lu, Huancheng Lin, Taku Komura, and Minchen Li. 2025a. StiffGIPC: Advancing GPU IPC for Stiff Affine-Deformable Simulation. *ACM Transactions on Graphics* 44, 3 (2025), 1–20.
- Zizhou Huang, Maxwell Paik, Zachary Ferguson, Daniele Panozzo, and Denis Zorin. 2025b. Geometric Contact Potential. *ACM Transactions on Graphics (TOG)* 44, 4 (2025), 1–24.
- Michel Jean and Jean Jacques Moreau. 1992. Unilaterality and dry friction in the dynamics of rigid body collections. In *1st Contact Mechanics International Symposium*. 31–48.
- Yupeng Jiang, Yidong Zhao, Clarence E Choi, and Jinhyun Choo. 2022. Hybrid continuum–discrete simulation of granular impact dynamics. *Acta Geotechnica* 17, 12 (2022), 5597–5612.
- Zhongshi Jiang, Scott Schaefer, and Daniele Panozzo. 2017. Simplicial complex augmentation framework for bijective maps. *ACM Transactions on Graphics* 36, 6 (2017).
- Couro Kane, Jerrold E Marsden, Michael Ortiz, and Matthew West. 2000. Variational integrators and the Newmark algorithm for conservative and dissipative mechanical systems. *International Journal for numerical methods in engineering* 49, 10 (2000), 1295–1325.
- Couro Kane, Eduardo A Repetto, Michael Ortiz, and Jerrold E Marsden. 1999. Finite element analysis of nonsmooth contact. *Computer methods in applied mechanics and engineering* 180, 1-2 (1999), 1–26.
- Tero Karras. 2012. Maximizing parallelism in the construction of BVHs, octrees, and k-d trees. In *Proceedings of the Fourth ACM SIGGRAPH/Eurographics Conference on High-Performance Graphics*. 33–37.
- Danny M Kaufman, Shinjiro Sueda, Doug L James, and Dinesh K Pai. 2008. Staggered projections for frictional contact in multibody systems. In *ACM SIGGRAPH Asia 2008 papers*. 1–11.
- Danny M Kaufman, Rasmus Tamstorf, Breannan Smith, Jean-Marie Aubry, and Eitan Grinspun. 2014. Adaptive nonlinearity for collisions in complex rod assemblies. *ACM Transactions on Graphics (TOG)* 33, 4 (2014), 1–12.
- Theodore Kim and David Eberle. 2020. Dynamic deformables: implementation and production practicalities. In *Acm siggraph 2020 courses*. 1–182.
- Lei Lan, Danny M Kaufman, Minchen Li, Chenfanfu Jiang, and Yin Yang. 2022a. Affine body dynamics: Fast, stable & intersection-free simulation of stiff materials. *arXiv preprint arXiv:2201.10022* (2022).
- Lei Lan, Minchen Li, Chenfanfu Jiang, Huamin Wang, and Yin Yang. 2023. Second-order Stencil Descent for Interior-point Hyperelasticity. *ACM Trans. Graph.* 42, 4 (2023).
- Lei Lan, Zixuan Lu, Jingyi Long, Chun Yuan, Xuan Li, Xiaowei He, Huamin Wang, Chenfanfu Jiang, and Yin Yang. 2024. Efficient GPU cloth simulation with non-distance barriers and subspace reuse. *arXiv preprint arXiv:2403.19272* (2024).
- Lei Lan, Guanqun Ma, Yin Yang, Changxi Zheng, Minchen Li, and Chenfanfu Jiang. 2022b. Penetration-free projective dynamics on the GPU. *ACM Transactions on Graphics (TOG)* 41, 4 (2022), 1–16.

Lei Lan, Yin Yang, Danny Kaufman, Junfeng Yao, Minchen Li, and Chenfanfu Jiang. 2021. Medial IPC: accelerated incremental potential contact with medial elastics. *ACM Transactions on Graphics* 40, 4 (2021).

Cheng Li, Min Tang, Ruofeng Tong, Ming Cai, Jieyi Zhao, and Dinesh Manocha. 2020b. P-cloth: interactive complex cloth simulation on multi-GPU systems using dynamic matrix assembly and pipelined implicit integrators. *ACM Transactions on Graphics (TOG)* 39, 6 (2020), 1–15.

Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy R Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M Kaufman. 2023b. Convergent Incremental Potential Contact. *arXiv preprint arXiv:2307.15908* (2023).

Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy R Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M Kaufman. 2020a. Incremental potential contact: intersection and inversion-free, large-deformation dynamics. *ACM Trans. Graph.* 39, 4 (2020), 49.

Minchen Li, Danny M Kaufman, and Chenfanfu Jiang. 2021. Codimensional incremental potential contact. *ACM Transactions on Graphics (TOG)* 40, 4 (2021), 1–24.

Xuan Li, Yu Fang, Lei Lan, Huamin Wang, Yin Yang, Minchen Li, and Chenfanfu Jiang. 2023a. Subspace-preconditioned gpu projective dynamics with contact for cloth simulation. In *SIGGRAPH Asia 2023 Conference Papers*. 1–12.

Xuan Li, Yu Fang, Minchen Li, and Chenfanfu Jiang. 2022. BFEMP: Interpenetration-free MPM-FEM coupling with barrier contact. *Computer Methods in Applied Mechanics and Engineering* 390 (2022), 114350.

Xuan Li, Minchen Li, Xuchen Han, Huamin Wang, Yin Yang, and Chenfanfu Jiang. 2024. A dynamic duo of finite elements and material points. In *ACM SIGGRAPH 2024 Conference Papers*. 1–11.

Miles Macklin, Kenny Erleben, Matthias Müller, Nuttapon Chentanez, Stefan Jeschke, and Viktor Makoviyshuk. 2019. Non-smooth newton methods for deformable multi-body dynamics. *ACM Transactions on Graphics (TOG)* 38, 5 (2019), 1–20.

Yuqi Meng, Yihao Shi, Kemeng Huang, Ning Guo, Taku Komura, Yin Yang, and Minchen Li. 2025. A B-Spline Finite Element Method for Cloth Simulation. *arXiv preprint arXiv:2506.18867* (2025).

Marek Krzysztof Misztal and Jakob Andreas Bærentzen. 2012. Topology-adaptive interface tracking using the deformable simplicial complex. *ACM Transactions on Graphics (TOG)* 31, 3 (2012), 1–12.

Matthias Müller, Nuttapon Chentanez, Tae-Yong Kim, and Miles Macklin. 2015. Air meshes for robust collision handling. *ACM Transactions on Graphics (ToG)* 34, 4 (2015), 1–9.

Rahul Narain, Armin Samii, and James F O'brien. 2012. Adaptive anisotropic remeshing for cloth simulation. *ACM transactions on graphics (TOG)* 31, 6 (2012), 1–10.

Jorge Nocedal and Stephen J. Wright. 2006. *Numerical Optimization* (2nd ed.). Springer Science & Business Media.

Miguel A Otaduy, Rasmus Tamstorf, Denis Steinemann, and Markus Gross. 2009. Implicit contact handling for deformable objects. In *Computer Graphics Forum*, Vol. 28. Wiley Online Library, 559–568.

Xavier Provot. 1997. Collision and self-collision handling in cloth model dedicated to design garments. In *Computer Animation and Simulation '97: Proceedings of the Eurographics Workshop in Budapest, Hungary, September 2–3, 1997*. Springer, 177–189.

Josua Sassen, Henrik Schumacher, Martin Rumpf, and Keenan Crane. 2024. Repulsive shells. *ACM Transactions on Graphics* 43, 4 (2024), 1–22.

Xing Shen, Runyuan Cai, Mengxiao Bi, and Tangjie Lv. 2024. Preconditioned nonlinear conjugate gradient method for real-time interior-point hyperelasticity. In *ACM SIGGRAPH 2024 Conference Papers*. 1–11.

Breannan Smith, Fernando De Goes, and Theodore Kim. 2018. Stable neo-hookean flesh simulation. *ACM Transactions on Graphics (TOG)* 37, 2 (2018), 1–15.

Paweł Strzelecki and Heiko von der Mosel. 2013. Tangent-point repulsive potentials for a class of non-smooth m-dimensional sets in $\mathbb{R}^n$. Part I: Smoothing and self-avoidance effects. *Journal of Geometric Analysis* 23, 3 (2013), 1085–1139.

Min Tang, Tongtong Wang, Zhongyuan Liu, Ruofeng Tong, and Dinesh Manocha. 2018. I-Cloth: Incremental collision handling for GPU-based interactive cloth simulation. *ACM Transactions on Graphics (TOG)* 37, 6 (2018), 1–10.

Ty Trusty, Yun Fei, David Levin, and Danny Kaufman. 2024. Trading Spaces: Adaptive Subspace Time Integration for Contacting Elastodynamics. *ACM Transactions on Graphics (TOG)* 43, 6 (2024), 1–16.

Mickeal Verschoor and Andrei C Jalba. 2019. Efficient and accurate collision response for elastically deformable models. *ACM Transactions on Graphics (TOG)* 38, 2 (2019), 1–20.

Bolun Wang, Zachary Ferguson, Teseo Schneider, Xin Jiang, Marco Attene, and Daniele Panozzo. 2021. A large-scale benchmark and an inclusion-based algorithm for continuous collision detection. *ACM Transactions on Graphics (TOG)* 40, 5 (2021), 1–16.

Tianyu Wang, Jiong Chen, Dongping Li, Xiaowei Liu, Huamin Wang, and Kun Zhou. 2023. Fast GPU-based two-way continuous collision handling. *ACM Transactions on Graphics* 42, 5 (2023), 1–15.

Peter Wriggers. 1995. Finite element algorithms for contact problems. *Archives of computational methods in engineering* 2, 4 (1995), 1–49.

Botao Wu, Zhendong Wang, and Huamin Wang. 2022. A GPU-based multilevel additive schwarz preconditioner for cloth and deformable body simulation. *ACM Transactions on Graphics (TOG)* 41, 4 (2022), 1–14.

Tianyi Xie, Minchen Li, Yin Yang, and Chenfanfu Jiang. 2023. A contact proxy splitting method for Lagrangian solid-fluid coupling. *ACM Transactions on Graphics (TOG)* 42, 4 (2023), 1–14.

A Frictional Contact

Our framework supports the same semi-implicit friction model as in IPC [Li et al. 2020a]. Specifically, we include the smoothed friction potential

$$U_f(\mathbf{x}) = \sum_i \mu_f F_i^t f_0(\|\mathbf{u}_i\|; \epsilon_v) \quad (22)$$

in the total potential energy $U(\mathbf{x})$, where i sums over all active contacts from the previous timestep, μ_f is the friction coefficient, F_i^t is the normal contact force at the previous timestep, and $\mathbf{u}_i$ is the tangential relative displacement. The ϵ_v -smoothed norm $f_0(\cdot; \epsilon_v)$ is a piecewise polynomial that satisfies $f_0(\|\mathbf{u}_i\|; \epsilon_v) = \|\mathbf{u}_i\|$ when $\|\mathbf{u}_i\| > h\epsilon_v$, and remains C^2 -continuous within the region $\|\mathbf{u}_i\| \leq h\epsilon_v$ (see [Li et al. 2020a] for details). This friction model is semi-implicit in the sense that both F_i^t and the tangent operator are determined in the previous timestep and treated as constant during the current solve. We calculate the contact force via

$$F_i^t \leftarrow h^{-2} \mu(c_i(\mathbf{x}^t) - s_i - \lambda_i/\mu), \quad (23)$$

where the scaling factor h^{-2} compensates for the dimensional difference between the contact penalty (Equation 12) and the potential energy $U(\mathbf{x})$. Note that the decay factor γ_i is omitted, as any constraint with $\gamma_i < 1$ necessarily yields $s_i > 0$, and therefore $c_i(\mathbf{x}^t) - s_i - \lambda_i/\mu = 0$.

B Comparison Details

B.1 Comparison Setup

For better reproducibility of our comparative experiments, we provide the setup details of the compared methods together with additional timing breakdowns omitted from the main paper. All comparisons are conducted on a desktop PC with an Intel Core i9-13900K CPU (24 cores), 64 GB RAM, and an NVIDIA GeForce RTX 4090 GPU, the same hardware used for our method. We use the following official GPU implementations for comparison:

- *GIPC*: https://github.com/KemengHuang/GPU_IPC (commit 405c1cc).
- *Cubic Barrier*: <https://github.com/st-tech/ppf-contact-solver> (commit 919539a).
- *OGC*: <https://github.com/newton-physics/newton> (commit 56c25d1).

We made minor modifications on GIPC and Cubic Barrier to support the energy models used in our testcases. Scripts and assets to reproduce the comparison test cases are available in our supplementary materials.

B.2 Detailed Performance Profiling for OGC

As OGC does not provide timing breakdown by default, we employ Warp's built-in ScopedTimer with full CUDA synchronization each frame to record the kernel-level timing information.

Kernel	Twisting Cloth (Easy)		Stacked Cloth (Easy)	
	Count	Time (ms)	Count	Time (ms)
memset	13206	38.888	1956	9.257
compute_tri_aabbs	161	0.522	61	0.720
memset_kernel	4	0.011	4	0.012
compute_total_bounds	2	0.010	2	0.025
compute_total_inv_edges	2	0.005	2	0.005
compute_morton_codes	2	0.005	2	0.012
memcpy DtoD	164	0.435	62	0.450
compute_key_deltas	2	0.006	2	0.008
build_leaves	2	0.006	2	0.057
build_hierarchy	2	0.063	2	0.530
mark_packed_leaf_nodes	2	0.008	2	0.049
compute_edge_aabbs	161	0.503	61	1.108
apply_rotation	80	0.386	—	—
bvh_refit_kernel	320	5.868	120	13.984
memtile_value_kernel	480	1.559	180	23.023
vertex_triangle_collision_detection_no_triangle_buffers_kernel	160	35.468	60	164.627
edge_colliding_edges_detection_kernel	160	57.493	60	757.425
compute_particle_conservative_bound	160	0.948	60	3.411
forward_step_penetration_free	80	0.325	30	0.497
accumulate_contact_force_and_hessian	19200	272.613	4500	4945.037
solve_trimesh_with_self_contact_penetration_free_tile	19200	175.882	4500	553.361
copy_particle_positions_back	19200	66.124	4500	25.634
update_velocity	80	0.263	30	0.224
Total CUDA time	72830	657.400	16198	6499.458
Total CUDA time (w/ graph)		515.374		6448.251

Table 3. CUDA kernel profiling comparison for OGC on Twisting cloth (easy, $N = 80$) and Stacked cloth (easy, $N = 30$).

By default, OGC enables Warp’s computation graph optimization to improve GPU utilization. To perform detailed kernel-level profiling, we temporarily disable the computation graph optimization, which incurs approximately a 27% performance overhead in the smaller test case (Twisting Cloth) and a negligible overhead in the larger case (Stacked Cloth). This modification is applied solely for obtaining detailed timing breakdowns here, and the computation graph optimization remains enabled in our main comparisons (§6.4).

We profile representative frames whose runtimes are close to the average values reported in Table 1, using the following settings:

- **Figure 22 (easy):** Twisting Cloth with $N = 80$ substeps and $N = 80$ iterations per substep (a total of 6,400 VBD iterations per timestep).
- **Figure 23 (easy):** Stacked Cloth with $N = 30$ substeps and $N = 30$ iterations per substep (a total of 900 VBD iterations per timestep).

Table 3 presents detailed performance profiling results for each CUDA kernel of OGC. Note that the kernels for Hessian assembly and per-vertex solving are executed $\#C \times \#\text{substeps} \times \#\text{iters}$ times, where $\#C$ denotes the number of mesh color groups.